AWS · CLOUD INFRASTRUCTURE · NETWORKING

AWS 인프라 & 네트워크 완전 학습 가이드

네트워크 기초부터 VPC 심화, 컴퓨팅·스토리지·데이터베이스,
보안·모니터링·IaC, 고가용성 아키텍처와 실습까지

- 처음 시작하는 사람을 위한 기초 개념 → 실무 수준의 세부 동작 원리
- 그림·비교표·CLI/Terraform 예제·체크리스트·장별 확인 문제 수록
- SAA / ANS(Advanced Networking) 자격증 대비 핵심 포인트 표시

개인 학습용 교재 · 2026년 11월 판

※ AWS 서비스 한도·가격·세부 기능은 수시로 바뀝니다. 실무 적용 전 공식 문서(docs.aws.amazon.com)로 최신 값을 확인하세요.

목차

0장 이 교재를 공부하는 방법
학습 순서, 실습 계정 준비, 비용 사고 방지

1장 클라우드와 AWS 개요
리전·AZ·엣지, 공동 책임 모델, Well-Architected

2장 네트워크 기초 (AWS를 위한)
OSI/TCP-IP, IP·CIDR·서브넷팅, 라우팅, NAT, DNS, TCP/UDP, HTTP/TLS

3장 IAM과 계정 관리
사용자·그룹·역할·정책, 정책 평가 로직, STS, Organizations·SCP

4장 VPC 기본
CIDR 설계, 서브넷, 라우트 테이블, IGW, NAT GW, SG vs NACL

5장 VPC 심화 & 하이브리드 네트워크
엔드포인트, PrivateLink, Peering, Transit Gateway, VPN, Direct Connect, Flow Logs, IPv6

6장 컴퓨팅
EC2, AMI, EBS, 구매 옵션, Auto Scaling, Lambda, ECS/EKS/Fargate

7장 로드 밸런싱
ALB, NLB, GWLB, 대상 그룹, 헬스 체크, 고정 세션

8장 스토리지
S3, EBS, EFS, FSx, Storage Gateway, 데이터 전송

9장 데이터베이스
RDS, Aurora, DynamoDB, ElastiCache, 선택 기준

10장 DNS와 콘텐츠 전송
Route 53, CloudFront, Global Accelerator

11장 보안 서비스
KMS, Secrets Manager, ACM, WAF, Shield, GuardDuty 등

12장 모니터링·감사·운영
CloudWatch, CloudTrail, Config, Systems Manager

13장 애플리케이션 통합
SQS, SNS, EventBridge, Kinesis, API Gateway

14장 Infrastructure as Code
CloudFormation, CDK, Terraform 실전 예제

15장 고가용성·재해 복구·비용 최적화
RTO/RPO, DR 전략 4가지, 비용 절감 체크리스트

16장 실습: 3-Tier 웹 아키텍처 구축
단계별 CLI 실습과 패킷 흐름 따라가기

17장 트러블슈팅 가이드
"연결이 안 돼요" 체계적 진단법

부록 용어집 · 포트 번호 · CIDR 표 · 학습 로드맵 · 확인 문제 정답

CHAPTER 0

이 교재를 공부하는 방법

학습 순서

AWS는 서비스가 200개가 넘지만, 인프라 엔지니어가 반드시 알아야 할 핵심은 생각보다 적습니다. 이 교재는 **"네트워크 → 권한(IAM) → VPC → 그 위에 올라가는 서비스"** 순서로 구성되어 있습니다. 이유는 간단합니다. AWS에서 장애·보안사고·연결 문제의 80% 이상은 **네트워크 설정**과 **권한 설정**에서 발생하기 때문입니다.

단계	범위	목표 기간(권장)
1. 기초 다지기	0~3장: 클라우드 개념, 네트워크 기초, IAM	1~2주
2. 네트워크 핵심	4~5장: VPC 기본/심화 — **가장 중요**, 그림을 직접 그려보며 학습	2~3주
3. 서비스 이해	6~13장: 컴퓨팅, LB, 스토리지, DB, DNS/CDN, 보안, 모니터링, 통합	3~4주
4. 자동화·설계	14~15장: IaC, 고가용성/DR, 비용	1~2주
5. 실전	16~17장: 직접 구축하고 일부러 망가뜨려 보며 트러블슈팅	1~2주

실습 계정 준비 & 비용 사고 방지

1. **루트 계정 보호**: 가입 직후 루트 계정에 MFA를 설정하고, 루트 액세스 키는 절대 만들지 않습니다. 일상 작업은 IAM Identity Center 사용자(또는 IAM 사용자)로 합니다.
2. **예산 알림(AWS Budgets)**: 월 예산(예: 10 USD)을 만들고 80%/100% 도달 시 이메일 알림을 받도록 설정합니다.
3. **프리 티어 확인**: 프리 티어 범위는 계정 생성 시점에 따라 다릅니다(2025년 7월 이후 신규 계정은 크레딧 기반 모델). Billing 콘솔에서 본인 계정 조건을 확인하세요.
4. **비싼 "숨은 비용" 주의**: NAT Gateway(시간당 + GB당 과금), 퍼블릭 IPv4 주소(2024년 2월부터 사용 중인 모든 퍼블릭 IPv4에 시간당 과금), 켜놓은 RDS/EKS 클러스터, 삭제하지 않은 EBS 스냅샷, Elastic IP 미연결 상태.
5. **실습 후 정리**: 실습 리소스는 태그(`Project=study`)를 붙이고, 끝나면 Terraform/CloudFormation으로 한 번에 삭제하는 습관을 들입니다.

> **⚠ 경고: 액세스 키 유출**
>
> GitHub 등 공개 저장소에 액세스 키를 올리면 수 분 안에 봇이 탐지해 채굴용 인스턴스를 대량으로 띄웁니다. 키는 코드에 넣지 말고, 가능하면 장기 키 대신 IAM 역할·SSO(임시 자격 증명)를 사용하세요.

AWS CLI 설정

```
# 설치 확인
aws --version
# IAM Identity Center(SSO) 방식 — 권장
aws configure sso
aws sso login --profile study
# 현재 누구로 인증되어 있는지 확인 (가장 자주 쓰는 명령)
aws sts get-caller-identity --profile study
# 기본 리전 설정 (서울 리전)
export AWS_DEFAULT_REGION=ap-northeast-2
export AWS_PROFILE=study
```

ℹ 이 교재의 표기

💡 **팁**은 실무 요령, ⚠ **경고**는 자주 하는 실수, **ℹ 참고**는 배경 지식, 📝 **확인 문제**는 장 끝 복습 문제입니다(정답은 부록). `[SAA]`, `[ANS]` 는 해당 자격증 시험 빈출 포인트입니다.

CHAPTER 1

클라우드와 AWS 개요

학습 목표 · 클라우드 서비스 모델을 구분한다 · 리전/가용 영역/엣지 로케이션의 차이와 선택 기준을 설명한다 · 공동 책임 모델로 "누가 무엇을 지키는지" 판단한다 · Well-Architected 6대 원칙을 안다

1.1 클라우드 컴퓨팅이란

클라우드 컴퓨팅은 **컴퓨팅 자원(서버, 스토리지, 네트워크, DB 등)을 인터넷을 통해 필요할 때 빌려 쓰고 쓴 만큼 비용을 내는 모델**입니다. 온프레미스(자체 데이터센터)와 비교하면 다음과 같은 차이가 있습니다.

항목	온프레미스	클라우드(AWS)
초기 비용	서버·랙·네트워크 장비 구매(CAPEX)	없음, 사용량 기반(OPEX)
용량 계획	최대 피크 기준으로 미리 구매 → 유휴 자원 발생	수요에 따라 자동 확장/축소(Auto Scaling)
구축 속도	장비 조달에 수 주~수 개월	수 분
글로벌 확장	해외 IDC 계약 필요	리전 선택만으로 즉시
책임 범위	물리 보안부터 애플리케이션까지 전부	공동 책임 모델에 따라 분담

서비스 모델: IaaS / PaaS / SaaS

모델	사용자가 관리	AWS가 관리	AWS 예시
IaaS	OS, 런타임, 앱, 데이터, 네트워크 설정	물리 서버, 하이퍼바이저, 데이터센터	EC2, VPC, EBS
PaaS	애플리케이션 코드, 데이터	OS 패치, 런타임, 확장	Elastic Beanstalk, RDS, Lambda(FaaS)
SaaS	사용 설정, 데이터	전부	Amazon WorkMail, QuickSight

배포 모델은 **퍼블릭 클라우드**, **프라이빗 클라우드**, 그리고 둘을 연결한 **하이브리드 클라우드**(VPN/Direct Connect로 온프레미스와 AWS 연결)가 있습니다.

1.2 AWS 글로벌 인프라

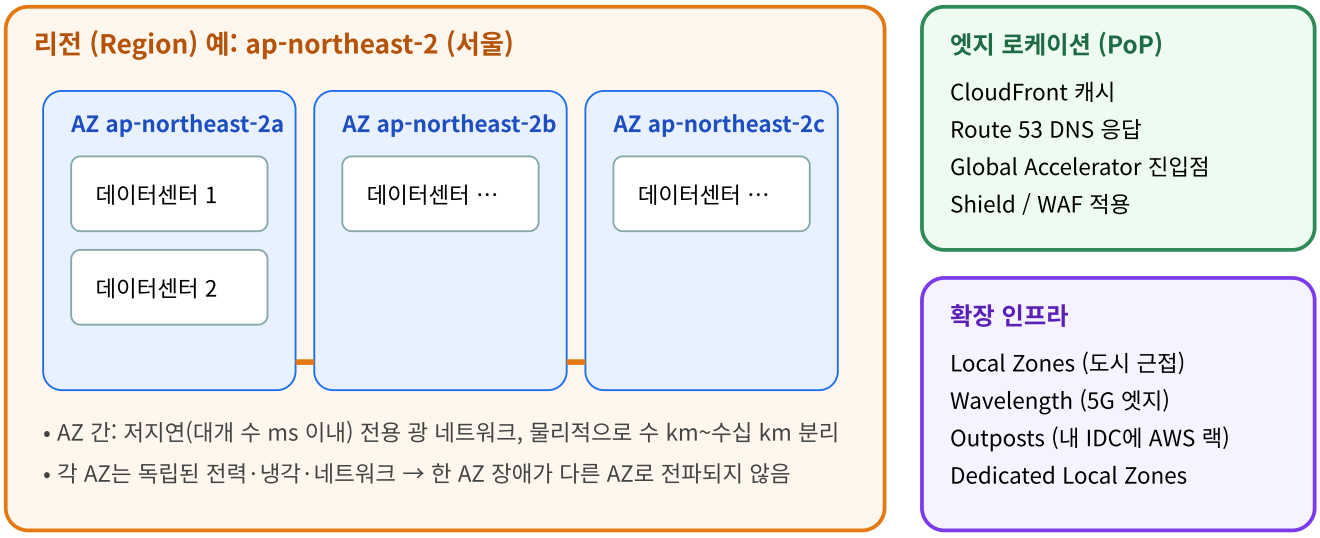


그림 1-1. 리전 – 가용 영역(AZ) – 데이터센터의 관계와 엣지 인프라

리전(Region)

리전은 지리적으로 분리된 지역으로, **최소 3개 이상의 가용 영역**으로 구성됩니다. 리전끼리는 완전히 독립되어 있어, 데이터는 사용자가 명시적으로 복제하지 않는 한 리전을 벗어나지 않습니다(데이터 주권). 한국에는 `ap-northeast-2` (서울) 리전이 있으며 4개의 AZ를 제공합니다.

리전 선택 기준 4가지

1. **규정 준수**: 개인정보·금융 데이터의 국외 이전 제한(데이터 주권)
2. **지연 시간**: 사용자와 가까운 리전일수록 응답이 빠름
3. **서비스 가용성**: 신규 서비스는 일부 리전에서 먼저 출시됨
4. **가격**: 리전별로 단가가 다름(일반적으로 us-east-1이 저렴한 편)

가용 영역(Availability Zone, AZ)

AZ는 하나 이상의 독립된 데이터센터 묶음입니다. **고가용성의 기본 단위**이며, 운영 서비스는 반드시 **2개 이상의 AZ**에 분산 배치해야 합니다.

> **⚠ AZ 이름 ≠ 물리 위치**
>
> `ap-northeast-2a` 라는 이름은 계정마다 실제 물리 AZ에 다르게 매핑될 수 있습니다(특정 AZ에 부하가 몰리는 것을 막기 위해). 여러 계정 간에 같은 물리 AZ를 맞춰야 할 때는 **AZ ID**(예: `apne2-az1`)를 사용하세요. `aws ec2 describe-availability-zones` 로 확인할 수 있습니다.

글로벌 서비스 vs 리전 서비스

범위	서비스
글로벌	IAM, Route 53, CloudFront, WAF(CloudFront용), Organizations, Global Accelerator
리전	VPC, S3(버킷은 리전에 생성되지만 이름은 전 세계 고유), DynamoDB, Lambda, ELB, RDS 등 대부분
AZ	EC2 인스턴스, EBS 볼륨, 서브넷, NAT Gateway(존 단위)

1.3 공동 책임 모델 (Shared Responsibility Model)

AWS는 **"클라우드 의 보안(Security OF the cloud)"**, 고객은 **"클라우드 안에서의 보안(Security IN the cloud)"**을 책임집니다.

AWS 책임	고객 책임
데이터센터 물리 보안, 하드웨어, 하이퍼바이저, 글로벌 네트워크 인프라, 관리형 서비스의 OS 패치(RDS, Lambda 등)	IAM 권한 관리, 데이터 암호화 설정, 보안 그룹/NACL, EC2 게스트 OS 패치, 애플리케이션 보안, 백업 정책

서비스가 관리형(추상화)일수록 고객 책임이 줄어듭니다. 예를 들어 EC2에서는 OS 패치가 고객 몫이지만, RDS에서는 AWS가 DB 엔진 패치를 수행합니다(시점 선택은 고객). 하지만 **데이터와 접근 권한은 어떤 서비스에서도 항상 고객 책임**입니다.

1.4 AWS Well-Architected Framework

AWS가 제시하는 좋은 아키텍처의 6대 원칙(Pillar)입니다. 설계 리뷰 시 체크리스트로 활용됩니다.

원칙	핵심 질문	대표 실천
운영 우수성	변경을 안전하게, 자주 배포하고 운영 상황을 파악하는가?	IaC, 작은 단위의 잦은 배포, 런북, 관측성
보안	자격 증명·데이터·네트워크를 계층별로 보호하는가?	최소 권한, 전 계층 보안, 암호화, 추적성
안정성	장애에서 자동으로 복구되는가?	Multi-AZ, 자동 복구, 백업, 한도(Quota) 관리
성능 효율성	요구에 맞는 자원을 효율적으로 쓰는가?	서버리스, 캐시, 적절한 인스턴스 유형
비용 최적화	불필요한 비용 없이 가치를 제공하는가?	Right-sizing, Savings Plans, 유휴 자원 제거
지속 가능성	환경 영향을 최소화하는가?	Graviton, 사용률 극대화, 관리형 서비스

📝 확인 문제 1

1) 서울 리전에 웹 서비스를 운영합니다. 한 데이터센터의 전력 장애에도 서비스가 유지되게 하려면 최소한 무엇을 해야 할까요?

2) EC2 위에 직접 설치한 MySQL의 보안 패치는 누구 책임일까요? RDS MySQL이라면?

3) 두 개의 AWS 계정에서 "같은 물리 AZ"에 리소스를 두려면 무엇을 기준으로 맞춰야 하나요?

CHAPTER 2

네트워크 기초 (AWS를 위한)

학습 목표 · OSI 7계층과 AWS 서비스를 매핑한다 · CIDR 표기를 읽고 서브넷을 직접 계산한다 · 라우팅 테이블의 "가장 긴 접두사 일치" 규칙을 이해한다 · NAT, DNS, TCP 핸드셰이크, TLS를 설명한다

2.1 OSI 7계층과 TCP/IP 모델

계층	이름	역할 / 데이터 단위	AWS에서 관련된 것
7	응용	HTTP, DNS, SMTP 등 / 메시지	ALB, CloudFront, WAF, API Gateway, Route 53
6	표현	암호화, 인코딩(TLS가 걸치는 영역)	ACM 인증서, TLS 종료
5	세션	세션 관리	—
4	전송	TCP/UDP, 포트 번호 / 세그먼트	NLB, 보안 그룹·NACL의 포트 규칙
3	네트워크	IP 주소, 라우팅 / 패킷	VPC, 서브넷, 라우트 테이블, TGW, GWLB
2	데이터링크	MAC 주소, 이더넷 / 프레임	ENI의 MAC (VPC는 L2 브로드캐스트를 지원하지 않음)
1	물리	전기·광 신호	Direct Connect의 광 회선

> **ℹ VPC는 "L3 네트워크"**
> VPC에는 브로드캐스트/멀티캐스트가 기본적으로 없습니다(멀티캐스트는 Transit Gateway에서 별도 지원). ARP 요청도 AWS 하이퍼바이저(Nitro)가 대신 응답합니다. 그래서 온프레미스의 VRRP/HSRP 같은 L2 기반 가용성 기법은 그대로 쓸 수 없고, 대신 로드밸런서나 라우트 테이블 변경, Elastic IP 재할당 방식을 씁니다.

2.2 IPv4 주소와 CIDR

IPv4 주소는 32비트이며 8비트씩 4개로 나눠 `10.0.1.25` 처럼 표기합니다. **CIDR(Classless Inter-Domain Routing)** 표기 `10.0.0.0/16` 에서 `/16` 은 "앞의 16비트가 네트워크 부분"이라는 뜻입니다. 나머지 16비트가 호스트 부분이므로 2^{16} = 65,536개의 주소를 가집니다.

```
10.0.0.0/16
  10       .0        .0        .0
  00001010 .00000000 ┆ 00000000 .00000000
   └──────── 네트워크 16비트 ────────┘  └──── 호스트 16비트 ────┘
  범위: 10.0.0.0 ~ 10.0.255.255  (65,536개)
서브넷 마스크로는 255.255.0.0
```

자주 쓰는 프리픽스 크기

CIDR	주소 수	AWS 서브넷에서 사용 가능	용도 예시
/16	65,536	65,531	VPC 최대 크기
/20	4,096	4,091	기본 VPC의 서브넷, EKS 파드용 큰 서브넷
/22	1,024	1,019	프라이빗 앱 서브넷
/24	256	251	일반적인 서브넷
/26	64	59	DB, 엔드포인트 서브넷
/28	16	11	AWS 서브넷 최소 크기, TGW 연결 서브넷
/32	1	—	단일 호스트(보안 그룹 규칙에서 특정 IP 지정)

공식: 주소 수 = $2^{(32 - \text{프리픽스})}$. 프리픽스가 1 커지면 크기는 절반이 됩니다.

사설 IP 대역 (RFC 1918)

대역	범위	비고
10.0.0.0/8	10.0.0.0 – 10.255.255.255	가장 큼, 대규모 기업망에서 주로 사용
172.16.0.0/12	172.16.0.0 – 172.31.255.255	AWS 기본 VPC가 172.31.0.0/16 사용
192.168.0.0/16	192.168.0.0 – 192.168.255.255	가정용 공유기에서 흔함 → VPN 충돌 주의

그 밖에 `100.64.0.0/10`(CGNAT, RFC 6598)도 VPC 보조 CIDR로 쓸 수 있으며, EKS에서 파드 IP 고갈 문제 해결용으로 자주 활용됩니다. `169.254.0.0/16` 은 링크 로컬로, AWS에서는 `169.254.169.254` (인스턴스 메타데이터), `169.254.169.253` (DNS) 등에 쓰입니다.

서브넷팅 실전 계산

문제: `10.10.0.0/16` VPC를 3개 AZ × (퍼블릭, 프라이빗 앱, 프라이빗 DB) = 9개 서브넷으로 나누되, 앱 서브넷을 가장 크게 하고 싶다.

```
# 앱 서브넷: /20 (4,096개) × 3
10.10.0.0/20    app-a   (10.10.0.0  ~ 10.10.15.255)
10.10.16.0/20   app-b   (10.10.16.0 ~ 10.10.31.255)
10.10.32.0/20   app-c   (10.10.32.0 ~ 10.10.47.255)
# 퍼블릭 서브넷: /24 (256개) × 3 — LB, NAT GW 정도만 들어가므로 작게
10.10.48.0/24   pub-a
10.10.49.0/24   pub-b
10.10.50.0/24   pub-c
# DB 서브넷: /24 × 3
10.10.52.0/24   db-a
10.10.53.0/24   db-b
10.10.54.0/24   db-c
# 10.10.64.0/18 이후는 미래 확장용으로 비워둠 (중요!)
```

> 💡 **서브넷 경계 빠르게 계산하기**
>
> /20의 블록 크기는 3번째 옥텟 기준 $2^{(24-20)}$ = 16입니다. 그래서 0, 16, 32, 48 …에서 시작합니다. /22면 4씩(0, 4, 8, …), /26이면 4번째 옥텟 기준 64씩(0, 64, 128, 192)입니다. 시작 주소가 블록 크기의 배수가 아니면 잘못된 CIDR입니다(예: `10.0.5.0/22` 는 무효, `10.0.4.0/22` 가 맞음).

2.3 라우팅의 원리

라우터(그리고 AWS 라우트 테이블)는 패킷의 **목적지 IP**를 보고 다음 홉(next hop)을 결정합니다. 여러 경로가 일치하면 **가장 긴 접두사 일치(Longest Prefix Match)** 규칙에 따라 더 구체적인 경로가 이깁니다.

```
라우트 테이블:
  10.0.0.0/16     → local
  10.1.0.0/16     → pcx-1234 (VPC Peering)
  10.1.5.0/24     → tgw-abcd (Transit Gateway)
  0.0.0.0/0       → igw-9999

목적지 10.1.5.7  → 10.1.0.0/16 과 10.1.5.0/24 둘 다 일치 → /24가 더 길므로 TGW로
목적지 8.8.8.8   → 0.0.0.0/0 만 일치 → 인터넷 게이트웨이로
목적지 10.0.3.4  → local (VPC 내부 통신)
```

`0.0.0.0/0` 은 "모든 주소"를 뜻하는 **기본 경로(default route)**입니다. 프리픽스 길이가 0이라 항상 가장 마지막 순위입니다.

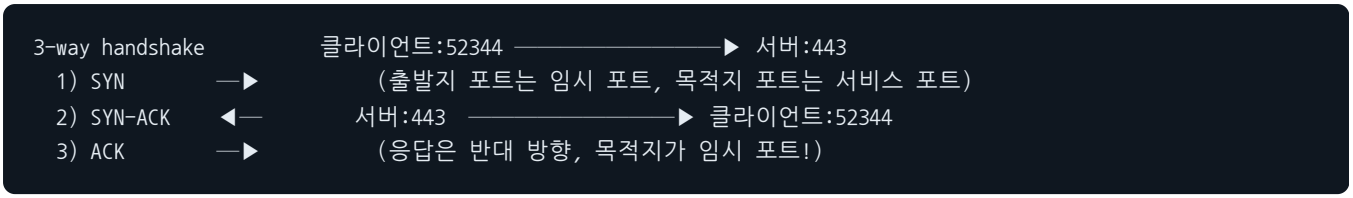
2.4 NAT (Network Address Translation)

사설 IP는 인터넷에서 라우팅되지 않으므로, 사설 IP를 가진 서버가 인터넷에 나가려면 공인 IP로 바꿔주는 장치가 필요합니다.

- **SNAT / PAT**: 여러 사설 IP를 하나의 공인 IP로 변환(포트로 구분). AWS의 **NAT Gateway**가 이 역할 → 프라이빗 서브넷의 아웃바운드 인터넷 접근.
- **1:1 NAT**: 사설 IP 하나와 공인 IP 하나를 고정 매핑. AWS의 **인터넷 게이트웨이(IGW)**가 퍼블릭 IP/EIP를 가진 인스턴스에 대해 이 역할을 수행합니다. 인스턴스 OS 안에서는 공인 IP가 보이지 않고 사설 IP만 보이는 이유가 이것입니다.

2.5 TCP와 UDP

구분	TCP	UDP
연결	연결 지향(3-way handshake)	비연결
신뢰성	순서 보장, 재전송, 흐름 제어	보장 없음, 빠름
예시	HTTP/1.1·2, SSH, DB 연결	DNS 질의, VoIP, 게임, QUIC(HTTP/3)
AWS LB	ALB(HTTP), NLB(TCP/TLS)	NLB(UDP)



> **⚠ 임시 포트(Ephemeral Port)와 NACL [SAA][ANS]**
>
> 클라이언트는 출발지 포트로 임시 포트(Linux 32768–60999, Windows 49152–65535, NAT GW/ELB는 1024–65535)를 씁니다. 서버의 응답은 이 임시 포트로 돌아가므로, **상태 비저장(stateless)인 NACL**은 아웃바운드에 `1024-65535` 범위를 허용해야 응답이 나갈 수 있습니다. 보안 그룹은 상태 저장(stateful)이라 신경 쓸 필요가 없습니다.

2.6 DNS

DNS는 도메인 이름을 IP 주소로 변환합니다. 브라우저에서 `www.example.com` 을 입력하면:

1. OS/브라우저 캐시 확인 → 없으면 **재귀 리졸버**(VPC에서는 Route 53 Resolver, VPC CIDR+2 주소)에 질의
2. 리졸버가 루트 서버 → `.com` TLD 서버 → `example.com` 의 **권한 있는(authoritative) 네임서버**(예: Route 53 호스팅 영역) 순서로 질의
3. 응답을 TTL 동안 캐시

레코드	의미
A / AAAA	이름 → IPv4 / IPv6
CNAME	이름 → 다른 이름 (Zone Apex, 즉 example.com 자체에는 사용 불가)
Alias (Route 53 전용)	이름 → AWS 리소스(ELB, CloudFront, S3 웹사이트 등). Zone Apex 가능, 조회 무료
MX / TXT / NS / SOA	메일 서버 / 텍스트(SPF·도메인 검증) / 네임서버 / 영역 정보
SRV / CAA / PTR	서비스 위치 / 인증서 발급 허용 CA / 역방향 조회

2.7 HTTP와 TLS

HTTPS = HTTP + TLS. TLS는 서버 인증서로 서버 신원을 검증하고, 세션 키를 교환해 트래픽을 암호화합니다. AWS에서는 **ACM**(무료 공인 인증서)을 ALB·CloudFront·API Gateway에 붙여 TLS를 종료하는 것이 일반적입니다. **SNI(Server Name Indication)** 덕분에 하나의 ALB 리스너에 여러 도메인의 인증서를 붙일 수 있습니다.

상태 코드	의미	AWS에서 흔한 원인
301/302	리다이렉트	ALB HTTP→HTTPS 리다이렉트 규칙
403	권한 없음	S3 버킷 정책, WAF 차단, CloudFront OAC 설정 오류
502	Bad Gateway	대상이 비정상 응답/연결 끊김, Lambda 응답 형식 오류
503	Service Unavailable	ALB 대상 그룹에 정상(healthy) 대상이 없음
504	Gateway Timeout	대상이 유휴 제한 시간(기본 60초) 내에 응답 안 함, SG가 LB→대상 차단

📝 확인 문제 2

1) `172.20.0.0/22` 에는 몇 개의 IP가 있고, AWS 서브넷으로 만들면 몇 개를 쓸 수 있나요?
2) `10.0.8.0/21` 은 올바른 CIDR인가요? `10.0.12.0/21` 은?
3) 라우트 테이블에 `10.0.0.0/8 → tgw`, `10.20.0.0/16 → pcx` 가 있을 때 `10.20.3.3` 은 어디로 가나요?
4) NACL 인바운드에 443만 허용하고 아웃바운드에 443만 허용했습니다. 외부 사용자의 HTTPS 접속은 될까요?

CHAPTER 3

IAM과 계정 관리

학습 목표 · IAM 주체(Principal)와 정책 구조를 읽고 쓴다 · 명시적 거부 → 허용 → 묵시적 거부의 평가 순서를 설명한다 · 역할(Role)과 STS AssumeRole 흐름을 이해한다 · 멀티 계정 구조(Organizations, SCP)를 설계한다

3.1 IAM 구성 요소

요소	설명	실무 원칙
루트 사용자	계정 생성 이메일. 모든 권한, 제한 불가(SCP 제외)	MFA 설정 후 금고에 보관. 결제·계정 해지 등 일부 작업에만 사용
IAM 사용자	장기 자격 증명(비밀번호, 액세스 키)을 가진 개체	사람은 Identity Center(SSO)로, 사용자 생성은 최소화
IAM 그룹	사용자 묶음. 그룹에 정책 연결	그룹 중첩 불가, 그룹은 주체가 될 수 없음
IAM 역할	누군가가 "맡는(assume)" 임시 권한 세트. 장기 키 없음	EC2·Lambda·크로스 계정·페더레이션은 모두 역할로
정책	권한을 정의한 JSON 문서	최소 권한 원칙, 관리형 정책 + 조건(Condition)

3.2 정책 문서 해부

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOwnBucketOnlyFromCorpNetwork",
      "Effect": "Allow",                        // Allow 또는 Deny
      "Action": ["s3:GetObject", "s3:ListBucket"],  // 서비스:API
      "Resource": [
        "arn:aws:s3:::my-study-bucket",         // ListBucket은 버킷 ARN
        "arn:aws:s3:::my-study-bucket/*"        // GetObject는 객체 ARN
      ],
      "Condition": {
        "IpAddress": { "aws:SourceIp": "203.0.113.0/24" },
        "Bool": { "aws:SecureTransport": "true" }
      }
    }
  ]
}
```

ARN 형식: `arn:partition:service:region:account-id:resource` . 예: `arn:aws:ec2:ap-northeast-2:123456789012:instance/i-0abc` . IAM·S3처럼 글로벌 성격의 서비스는 region/account 부분이 비어 있을 수 있습니다.

정책의 종류

정책 유형	붙는 곳	역할
자격 증명 기반 정책	사용자, 그룹, 역할	"이 주체는 무엇을 할 수 있다" — 권한 부여
리소스 기반 정책	S3 버킷, SQS, KMS 키, Lambda 등	"누가 이 리소스에 접근할 수 있다" — `Principal` 필드 있음, 크로스 계정 허용 가능
권한 경계(Permissions Boundary)	사용자, 역할	부여 가능한 최대 권한의 상한선(권한 부여 아님)
SCP / RCP	Organizations OU·계정	계정 전체의 최대 권한 상한선(주체 기준 / 리소스 기준)
세션 정책	AssumeRole 호출 시 전달	해당 세션 권한을 추가로 축소
ACL	S3(레거시) 등	가급적 비활성화(S3 Object Ownership: Bucket owner enforced)

3.3 정책 평가 로직 [SAA]

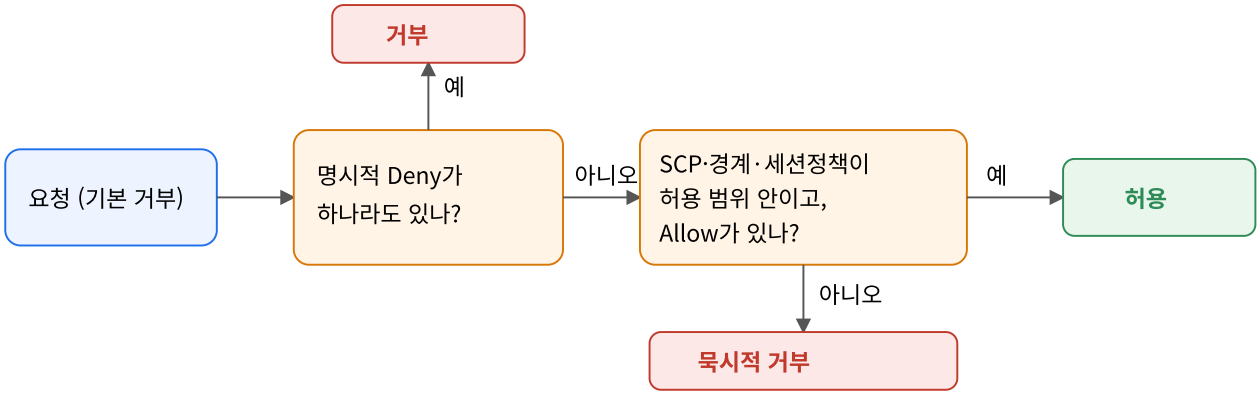


그림 3-1. 단순화한 IAM 정책 평가 흐름: 명시적 Deny가 항상 이긴다

- **기본은 거부**(루트 제외). 허용하려면 반드시 Allow가 있어야 합니다.
- **명시적 Deny는 모든 Allow를 이긴다.**
- SCP·권한 경계·세션 정책은 "교집합"으로 작동합니다. 이들 자체는 권한을 주지 않습니다.
- 같은 계정 안에서는 자격 증명 정책 *또는* 리소스 정책 중 하나만 허용해도 됩니다. **크로스 계정**에서는 **양쪽 모두** (호출자 계정의 자격 증명 정책 + 대상 리소스 정책) 허용해야 합니다.

3.4 역할(Role)과 STS

역할에는 두 가지 정책이 붙습니다. **신뢰 정책(Trust Policy)**은 "누가 이 역할을 맡을 수 있는가", **권한 정책**은 "맡은 뒤 무엇을 할 수 있는가"입니다.

```
// 신뢰 정책: EC2 서비스가 이 역할을 맡을 수 있음
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
```

EC2에 역할을 붙이면 **인스턴스 프로파일**을 통해 인스턴스 메타데이터(`http://169.254.169.254`)에서 임시 자격 증명이 자동으로 발급·갱신됩니다. SDK와 CLI는 이를 자동으로 사용합니다.

> **⚠ IMDSv2 강제**
>
> SSRF 취약점으로 메타데이터의 자격 증명이 탈취되는 사고(대표적으로 2019년 Capital One 사건)를 막기 위해, 토큰 기반의 **IMDSv2만 허용**(`HttpTokens=required`)하고 홉 제한을 1~2로 설정하세요. 신규 인스턴스 기본값으로 계정 수준에서 강제할 수 있습니다.

```
# IMDSv2로 메타데이터 조회
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
        -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
     http://169.254.169.254/latest/meta-data/iam/security-credentials/
# 크로스 계정 역할 맡기
aws sts assume-role --role-arn arn:aws:iam::222233334444:role/ReadOnly \
    --role-session-name study --external-id my-ext-id
```

External ID는 제3자 SaaS가 고객 계정의 역할을 맡을 때 "혼동된 대리인(confused deputy)" 문제를 막기 위해 신뢰 정책 조건에 넣는 값입니다.

3.5 멀티 계정 전략: Organizations, SCP, Control Tower

규모가 커지면 "하나의 계정에 모든 것"이 아니라 **계정을 경계로 분리**합니다. 계정은 가장 강력한 격리 단위(권한·한도·비용·장애 범위)입니다.

```
Root (관리 계정: 결제 · 조직 관리만, 워크로드 금지)
  ├── OU: Security     → Log Archive 계정, Security Tooling(Audit) 계정
  ├── OU: Infrastructure → Network 계정(TGW, DX, 공용 VPC), Shared Services
  ├── OU: Workloads
  │     ├── OU: Prod   → app-prod 계정 …
  │     └── OU: Dev    → app-dev 계정 …
  └── OU: Sandbox      → 개인 실험 계정 (예산 제한)
```

```
// SCP 예: 허용된 리전(서울, 버지니아) 외에서는 모든 작업 거부 (글로벌 서비스 예외)
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOutsideAllowedRegions",
    "Effect": "Deny",
    "NotAction": ["iam:*", "organizations:*", "route53:*", "cloudfront:*",
                  "sts:*", "support:*", "budgets:*", "waf:*"],
    "Resource": "*",
    "Condition": { "StringNotEquals": {
      "aws:RequestedRegion": ["ap-northeast-2", "us-east-1"] } }
  }]
}
```

- **SCP는 관리 계정에는 적용되지 않습니다.** 그래서 관리 계정에는 워크로드를 두지 않습니다.
- **Control Tower**: 위 구조(랜딩 존), 가드레일, 계정 자동 발급(Account Factory)을 자동 구성해주는 서비스.
- **IAM Identity Center**: 사내 IdP(Entra ID, Okta, Google 등)와 연동해 여러 계정에 SSO. 권한 세트(Permission Set)가 각 계정의 역할로 배포됩니다.

> 💡 **최소 권한을 찾는 도구**
>
> **IAM Access Analyzer**: 외부에 공유된 리소스 탐지, CloudTrail 기반 정책 생성, 미사용 권한·역할 탐지. **마지막 액세스 정보(Last accessed)**로 쓰지 않는 서비스 권한을 제거하세요.

> 📝 **확인 문제 3**
>
> 1) 사용자 정책에 `s3:*` Allow, 그룹 정책에 `s3:DeleteBucket` Deny가 있습니다. 버킷 삭제가 가능한가요?
> 2) 계정 A의 Lambda가 계정 B의 S3 버킷을 읽으려 합니다. 어디에 무엇을 설정해야 하나요(두 가지 방법)?
> 3) SCP에서 EC2를 허용했지만 사용자에게 아무 정책도 없습니다. 이 사용자는 EC2를 쓸 수 있나요?

CHAPTER 4

VPC 기본

학습 목표 · VPC CIDR을 겹치지 않게 설계한다 · 퍼블릭/프라이빗 서브넷의 정확한 정의를 말한다 · IGW, NAT Gateway를 통한 패킷 흐름을 그린다 · 보안 그룹과 NACL을 구분해 적용한다

4.1 VPC란

VPC(Virtual Private Cloud)는 AWS 안에 만드는 논리적으로 격리된 나만의 네트워크입니다. 하나의 리전에 속하며, 그 리전의 모든 AZ에 걸쳐 존재합니다. 서브넷은 하나의 AZ에만 속합니다.

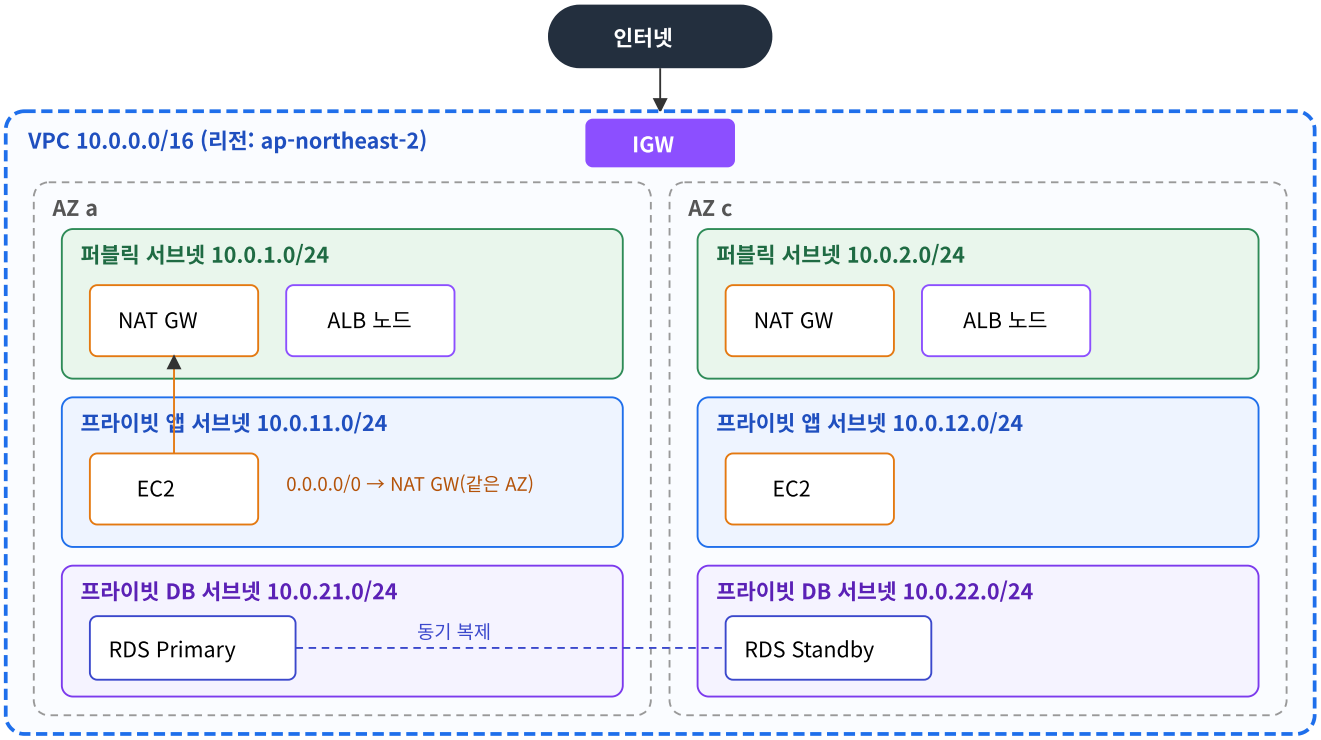


그림 4-1. 2개 AZ에 걸친 표준 3계층 VPC 구조

VPC CIDR 규칙

- IPv4 CIDR 크기는 **/16 ~ /28**. 기본 CIDR 외에 **보조 CIDR**을 추가할 수 있습니다(기본 한도 5개, 증가 요청 가능).
- 한 번 만든 기본 CIDR은 변경 불가 → **처음부터 넉넉하고 겹치지 않게** 설계하는 것이 매우 중요합니다.
- Peering/TGW/VPN으로 연결될 가능성이 있는 모든 네트워크(다른 VPC, 온프레미스, 협력사)와 **CIDR이 겹치면 직접 라우팅이 불가**합니다.
- 대규모 조직은 **VPC IPAM**으로 계정·리전별 IP 풀을 중앙 관리합니다.

> 💡 **CIDR 설계 예시 (조직 전체)**
>
> `10.0.0.0/8` 을 나눠 사용: 온프레미스 `10.0.0.0/12` , AWS 서울 `10.16.0.0/12` (그 안에서 VPC마다 /16 또는 /20), AWS 버지니아 `10.32.0.0/12` … 이렇게 리전별로 요약(summarize) 가능하게 잡으면 라우팅 테이블이 단순해집니다.

4.2 서브넷

각 서브넷에서는 **5개의 IP가 예약**되어 사용할 수 없습니다. `10.0.1.0/24` 기준:

주소	용도
10.0.1.0	네트워크 주소
10.0.1.1	VPC 라우터
10.0.1.2	DNS 서버(Route 53 Resolver) — 정확히는 VPC 기본 CIDR의 베이스+2
10.0.1.3	향후 사용을 위해 예약
10.0.1.255	브로드캐스트 주소(VPC는 브로드캐스트 미지원이지만 예약)

퍼블릭 서브넷 vs 프라이빗 서브넷 — 정확한 정의

서브넷 자체에 "퍼블릭" 속성이 있는 것이 아닙니다. **연결된 라우트 테이블에 `0.0.0.0/0 → igw-xxx` 경로가 있으면 퍼블릭 서브넷**입니다. 인스턴스가 실제로 인터넷과 통신하려면 추가로 **퍼블릭 IP 또는 Elastic IP**가 있어야 합니다.

구분	기본 경로	인바운드(인터넷→)	아웃바운드(→인터넷)
퍼블릭	IGW	가능(퍼블릭 IP + SG 허용 시)	가능
프라이빗 (NAT)	NAT Gateway	불가	가능(NAT 경유)
격리(Isolated)	없음	불가	불가 (VPC 엔드포인트로만 AWS 서비스 접근)

4.3 라우트 테이블

- 모든 VPC에는 **메인 라우트 테이블**이 있고, 명시적으로 연결하지 않은 서브넷은 메인을 사용합니다. → 실수 방지를 위해 메인은 프라이빗 상태로 두고, 서브넷마다 **명시적으로 연결**하세요.
- `local` 경로(VPC CIDR)는 삭제할 수 없으며, VPC 안의 모든 서브넷은 기본적으로 서로 통신 가능합니다. (더 구체적인 경로로 미들박스를 경유하게 만드는 것은 가능)
- 하나의 서브넷은 하나의 라우트 테이블에만 연결되며, 하나의 라우트 테이블은 여러 서브넷에 연결될 수 있습니다.
- 대상(target) 종류: `local`, IGW, NAT GW, ENI/인스턴스, VPC Peering, TGW, VGW, 게이트웨이 엔드포인트(프리픽스 리스트), GWLB 엔드포인트, Egress-only IGW 등.
- **게이트웨이 라우트 테이블(Ingress Routing)**: IGW/VGW에 라우트 테이블을 연결해, 들어오는 트래픽을 방화벽 어플라이언스로 먼저 보낼 수 있습니다.

4.4 인터넷 게이트웨이 (IGW)

- VPC당 1개만 연결, 수평 확장·고가용성이 기본 제공(관리할 것 없음, 대역폭 제한 없음, 비용 없음).
- 퍼블릭 IPv4를 가진 인스턴스에 대해 사설 ↔ 공인 IP의 **1:1 NAT** 수행.
- IPv6에서는 NAT 없이 라우팅만 수행합니다.

4.5 NAT Gateway

특징	내용
배치	**퍼블릭 서브넷**에 생성(퍼블릭 NAT), Elastic IP 연결. 프라이빗 서브넷 라우트 테이블에 `0.0.0.0/0 → nat-xxx`
가용성	**AZ 단위 리소스**. AZ 장애 대비를 위해 AZ마다 하나씩 두고, 각 AZ 프라이빗 서브넷은 같은 AZ의 NAT를 사용 (AZ 간 데이터 전송 비용도 절감). 2025년 출시된 **리전 NAT Gateway** 모드는 하나의 NAT로 여러 AZ에 자동 확장되므로 신규 구성 시 검토하세요.
성능	5 Gbps에서 시작해 100 Gbps까지 자동 확장. 목적지(IP:포트:프로토콜)당 동시 연결 55,000개 — 초과 시 EIP 추가(최대 8개)
보안	보안 그룹을 붙일 수 없음(NACL로 제어). 인바운드로 시작되는 연결은 받지 않음
비용	시간당 요금 + 처리 GB당 요금 → S3/DynamoDB 트래픽은 게이트웨이 엔드포인트로 우회해 비용 절감
프라이빗 NAT	EIP 없이 사설 IP로 NAT. 겹치는 CIDR 간 통신, 온프레미스로 나갈 때 IP 대역 통일 등에 사용

> **ℹ NAT 인스턴스**
>
> EC2에 NAT를 직접 구성하는 레거시 방식. 비용은 저렴하지만 가용성·확장을 직접 관리해야 하며, **소스/대상 확인(Source/Dest Check)을 비활성화**해야 동작합니다. 시험에서는 "관리 부담이 적은" 답으로 NAT Gateway를 고릅니다.

4.6 보안 그룹 vs 네트워크 ACL [SAA][ANS 최빈출]

항목	보안 그룹 (SG)	네트워크 ACL (NACL)
적용 위치	ENI(인스턴스, LB, RDS, Lambda in VPC 등)	서브넷 경계
상태	**Stateful** — 허용된 요청의 응답은 자동 허용	**Stateless** — 응답 트래픽도 반대 방향 규칙 필요
규칙 종류	허용(Allow)만	허용과 거부(Deny) 모두
평가 방식	모든 규칙을 합쳐서 평가	규칙 번호 오름차순, 처음 일치한 규칙 적용 후 종료
기본값	새 SG: 인바운드 전부 거부, 아웃바운드 전부 허용	기본 NACL: 전부 허용 / 새로 만든 NACL: 전부 거부
소스 지정	CIDR, 프리픽스 리스트, **다른 보안 그룹 ID**	CIDR만
주 용도	주 방어선, 애플리케이션 계층 간 접근 제어	특정 IP 대역 차단, 서브넷 단위 가드레일

보안 그룹 체이닝 (참조)

IP 대신 **보안 그룹 ID를 소스로 지정**하면, Auto Scaling으로 IP가 바뀌어도 규칙을 수정할 필요가 없습니다.

```
sg-alb  : 인바운드 443  from 0.0.0.0/0
sg-app  : 인바운드 8080 from sg-alb      ← ALB에서 온 트래픽만
sg-db   : 인바운드 3306 from sg-app      ← 앱 서버에서 온 트래픽만
```

NACL 예시 (퍼블릭 웹 서브넷)

방향	번호	프로토콜/포트	소스·대상	동작
인바운드	90	ALL	198.51.100.0/24 (악성 대역)	DENY
인바운드	100	TCP 443	0.0.0.0/0	ALLOW
인바운드	110	TCP 1024-65535	0.0.0.0/0 (아웃바운드 요청의 응답)	ALLOW
아웃바운드	100	TCP 1024-65535	0.0.0.0/0 (클라이언트에게 응답)	ALLOW
아웃바운드	110	TCP 443	0.0.0.0/0 (패키지 다운로드 등)	ALLOW
양방향	*	ALL	0.0.0.0/0	DENY (기본, 삭제 불가)

> 💡 **규칙 번호는 10 또는 100 단위로**
> 나중에 사이에 규칙을 끼워 넣을 수 있도록 간격을 두세요. 거부 규칙은 허용 규칙보다 작은 번호여야 효과가 있습니다.

4.7 ENI, 퍼블릭 IP, Elastic IP

- **ENI(Elastic Network Interface)**: 가상 NIC. 사설 IP(주/보조), 퍼블릭 IP, MAC, 보안 그룹을 가짐. 같은 AZ의 다른 인스턴스로 옮겨 붙일 수 있음.
- **자동 할당 퍼블릭 IP**: 인스턴스 정지/시작 시 바뀜. 서브넷 설정 "퍼블릭 IPv4 자동 할당"으로 제어.
- **Elastic IP**: 고정 공인 IP. 계정에 할당되어 다른 인스턴스로 재매핑 가능(장애 조치). 리전당 기본 5개.
- **Source/Destination Check**: 기본 켜짐. 인스턴스가 자신의 IP가 아닌 트래픽을 송수신하면 차단 → NAT 인스턴스, 방화벽, VPN 어플라이언스는 꺼야 함.

4.8 VPC DNS 설정

- `enableDnsSupport` : VPC+2 주소의 Amazon DNS(Route 53 Resolver) 사용 여부. 꺼지면 프라이빗 호스팅 영역·인터페이스 엔드포인트 DNS가 동작하지 않습니다.
- `enableDnsHostnames` : 퍼블릭 IP가 있는 인스턴스에 퍼블릭 DNS 이름 부여. **인터페이스 엔드포인트의 프라이빗 DNS**를 쓰려면 두 옵션 모두 켜야 합니다.
- DHCP 옵션 세트로 도메인 이름, DNS 서버, NTP 서버를 지정할 수 있습니다.

4.9 기본 VPC

각 리전에는 `172.31.0.0/16` 의 기본 VPC가 있으며, 모든 서브넷이 퍼블릭(/20)이고 IGW가 연결되어 있습니다. 빠른 테스트에는 편하지만 **운영 환경에서는 사용하지 말고 직접 설계한 VPC를 사용**하세요.

📝 확인 문제 4

1) 라우트 테이블에 IGW 경로가 있는 서브넷에 퍼블릭 IP 없는 인스턴스를 띄웠습니다. 인터넷 접속이 될까요?
2) 프라이빗 서브넷 인스턴스들이 AZ-a에 있는 NAT GW 하나만 쓰고 있습니다. 어떤 문제가 있나요?
3) 특정 IP 대역(공격자)을 차단하려 합니다. 보안 그룹과 NACL 중 무엇을 써야 하나요? 이유는?
4) DB 보안 그룹의 소스를 앱 서버 IP 대신 무엇으로 지정하면 좋을까요?

CHAPTER 5

VPC 심화 & 하이브리드 네트워크

학습 목표 · 게이트웨이/인터페이스 엔드포인트와 PrivateLink를 구분한다 · Peering과 Transit Gateway의 차이와 선택 기준을 안다 · Site-to-Site VPN과 Direct Connect의 구성 요소(VGW, CGW, VIF, BGP)를 설명한다 · 하이브리드 DNS와 Flow Logs, IPv6 설계를 이해한다

5.1 VPC 엔드포인트

프라이빗 서브넷에서 S3, DynamoDB, SQS 같은 AWS 서비스를 쓸 때, 인터넷(NAT)을 거치지 않고 **AWS 내부 네트워크로 직접** 접근하게 해줍니다. 보안(인터넷 노출 없음)과 비용(NAT 처리 요금 절감) 두 가지 이점이 있습니다.

구분	게이트웨이 엔드포인트	인터페이스 엔드포인트 (PrivateLink)
지원 서비스	**S3, DynamoDB** 두 개뿐	대부분의 AWS 서비스 + 서드파티/자체 서비스 (S3도 지원)
동작 방식	라우트 테이블에 프리픽스 리스트(pl-xxx) 경로 추가	서브넷에 ENI(사설 IP) 생성, DNS로 해당 IP로 연결
비용	**무료**	AZ당 시간 요금 + GB당 요금
온프레미스/ 피어링에서 접근	불가(라우팅 기반이라 VPC 내부 전용)	가능(VPN/DX/TGW로 ENI IP에 도달)
보안 제어	엔드포인트 정책	엔드포인트 정책 + 보안 그룹

```
// S3 버킷 정책: 특정 VPC 엔드포인트를 통한 접근만 허용
{
  "Effect": "Deny", "Principal": "*", "Action": "s3:*",
  "Resource": ["arn:aws:s3:::secure-bucket", "arn:aws:s3:::secure-bucket/*"],
  "Condition": { "StringNotEquals": { "aws:SourceVpce": "vpce-0a1b2c3d" } }
}
```

⚠ 엔드포인트 사용 시 흔한 실수

- 게이트웨이 엔드포인트는 **같은 리전**의 S3에만 적용됩니다. 다른 리전 버킷은 여전히 NAT를 탑니다.
- 인터페이스 엔드포인트의 보안 그룹에서 **클라이언트 → 443 인바운드**를 허용하지 않아 타임아웃 나는 경우가 매우 흔합니다.
- 완전 격리 서브넷에서 SSM Session Manager를 쓰려면 `ssm`, `ssmmessages`, (필요 시 `ec2messages`) 인터페이스 엔드포인트가 필요합니다. ECR에서 이미지를 받으려면 `ecr.api`, `ecr.dkr` + S3 게이트웨이 엔드포인트가 필요합니다.

PrivateLink로 내 서비스 공개하기

서비스 제공자는 **NLB(또는 GWLB)** 앞에 **엔드포인트 서비스**를 만들고, 소비자는 자기 VPC에 인터페이스 엔드포인트를 만듭니다. 트래픽은 단방향(소비자 → 제공자)이고, **양쪽 CIDR이 겹쳐도** 동작하며, VPC 전체를 연결하지 않고 특정 서비스만 노출한다는 점이 Peering과의 큰 차이입니다. SaaS 제공, 멀티 계정 공용 서비스에 많이 씁니다. 최근에는 **VPC Lattice**(서비스 간 L7 연결·인증 관리형 서비스)와 **리소스 엔드포인트**(RDS 등 특정 리소스를 NLB 없이 공유)도 선택지입니다.

5.2 VPC Peering

- 두 VPC를 1:1로 연결. 같은/다른 계정, 같은/다른 리전 모두 가능. AWS 백본 사용, 대역폭 제한·단일 장애점 없음.
- **전이적 라우팅 불가(Non-transitive)**: A–B, B–C가 피어링되어 있어도 A는 C와 통신할 수 없습니다. B의 IGW/NAT/VPN을 빌려 쓰는 것(edge-to-edge)도 불가.
- **CIDR이 겹치면 생성 불가.**
- 양쪽 라우트 테이블에 상대 CIDR → `pcx-xxx` 경로를 직접 추가해야 하고, 보안 그룹도 허용해야 합니다(같은 리전이면 상대 VPC의 SG ID 참조 가능).
- VPC가 많아지면 연결 수가 n(n−1)/2로 폭증 → 10개 VPC면 45개. 이때 Transit Gateway를 씁니다.

5.3 Transit Gateway (TGW)

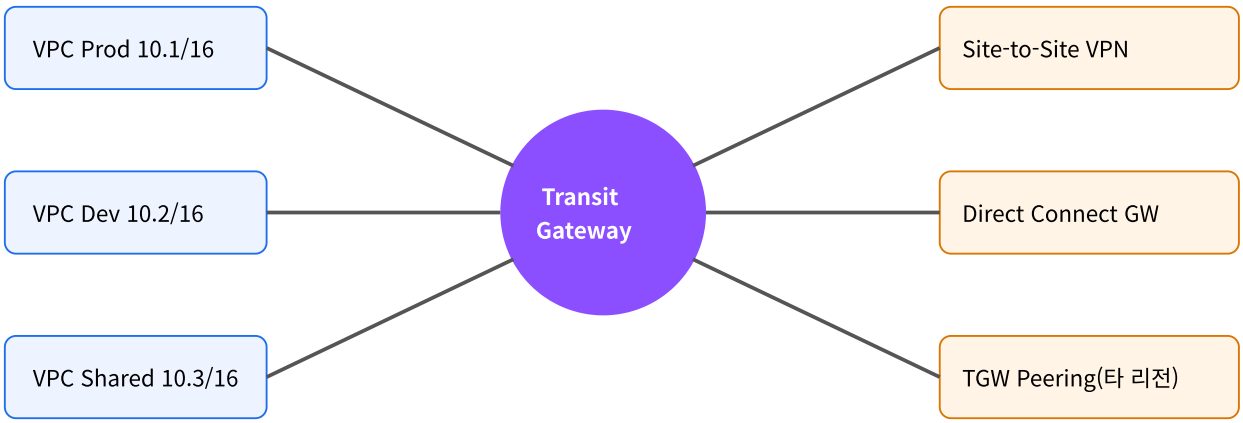


그림 5-1. 허브 앤 스포크: 모든 네트워크가 TGW 하나에 연결(Attachment)된다

개념	설명
Attachment	TGW에 붙는 연결: VPC, VPN, Direct Connect Gateway, TGW Peering, Connect(GRE/BGP, SD-WAN 연동)
TGW 라우트 테이블	TGW 내부의 라우팅 도메인. 여러 개 만들어 **세그먼트 분리**(예: Prod와 Dev는 서로 못 가고, 둘 다 Shared는 갈 수 있게)
Association	Attachment가 "어느 라우트 테이블을 보고 나갈지" 결정 (Attachment당 1개)
Propagation	Attachment의 CIDR을 어떤 라우트 테이블에 자동으로 전파(학습)할지 (여러 개 가능)
VPC Attachment 서브넷	AZ마다 하나의 서브넷 지정(관례적으로 전용 /28). 지정한 AZ에서만 트래픽 처리 가능
Appliance Mode	검사용 VPC(방화벽)에서 대칭 라우팅 보장 — 같은 흐름이 같은 AZ 방화벽을 지나게 함
성능	VPC attachment당 AZ별 최대 100 Gbps 버스트, VPN 터널당 1.25 Gbps(ECMP로 묶어 확장)

> 💡 **세그멘테이션 예시**
> RT-Prod: Prod VPC Attachment를 연결(Associate). 전파: Shared, 온프레미스(VPN/DX)
> RT-Dev: Dev VPC Attachment 연결. 전파: Shared
> RT-Shared: Shared VPC·VPN Attachment 연결. 전파: Prod, Dev, 온프레미스
> → Prod ↔ Dev는 서로의 경로가 없으므로 통신 불가. 블랙홀 경로(`blackhole`)로 명시적 차단도 가능.
> VPC 쪽 라우트 테이블에도 `10.0.0.0/8 → tgw-xxx` 같은 경로를 넣어야 한다는 점을 잊지 마세요.

대규모 글로벌 네트워크는 **AWS Cloud WAN**(정책 기반으로 여러 리전의 TGW·세그먼트를 중앙 관리)을 고려합니다.

5.4 Site-to-Site VPN

구성 요소	설명
CGW (Customer Gateway)	온프레미스 VPN 장비의 정보(공인 IP, BGP ASN)를 AWS에 등록한 리소스
VGW (Virtual Private Gateway)	VPC 하나에 붙는 AWS 측 VPN 종단점 (또는 TGW에 VPN을 붙임)
VPN 연결	**터널 2개**(서로 다른 AWS 엔드포인트, 고가용성), IPsec(IKEv1/v2)
라우팅	정적 라우팅 또는 **BGP(동적, 권장)** — BGP는 장애 시 자동 경로 전환
대역폭	터널당 최대 1.25 Gbps (Large 터널 옵션 시 5 Gbps). TGW + ECMP로 여러 터널 합산
Accelerated VPN	Global Accelerator 엣지를 통해 진입 → 원거리 지사 성능 개선 (TGW에서만)

- VPC 라우트 테이블에서 **라우트 전파(Route Propagation)**를 켜면 VGW가 학습한 온프레미스 경로가 자동으로 들어옵니다.
- 온프레미스 방화벽에서 UDP 500(IKE), UDP 4500(NAT-T), ESP(프로토콜 50)를 허용해야 합니다.
- 터널이 유휴 상태면 내려갈 수 있으므로 DPD 설정 또는 지속적인 핑으로 유지합니다.

- **Client VPN**: 개별 사용자 PC(OpenVPN 기반)가 VPC에 접속하는 관리형 원격 접속 VPN. 사이트 간 VPN과 구분하세요.

5.5 Direct Connect (DX)

온프레미스와 AWS를 **전용 물리 회선**으로 연결합니다. 인터넷을 거치지 않아 대역폭·지연이 일정하고, AWS에서 나가는 데이터 전송 비용이 인터넷보다 저렴합니다. 회선 개통에는 수 주가 걸릴 수 있습니다.

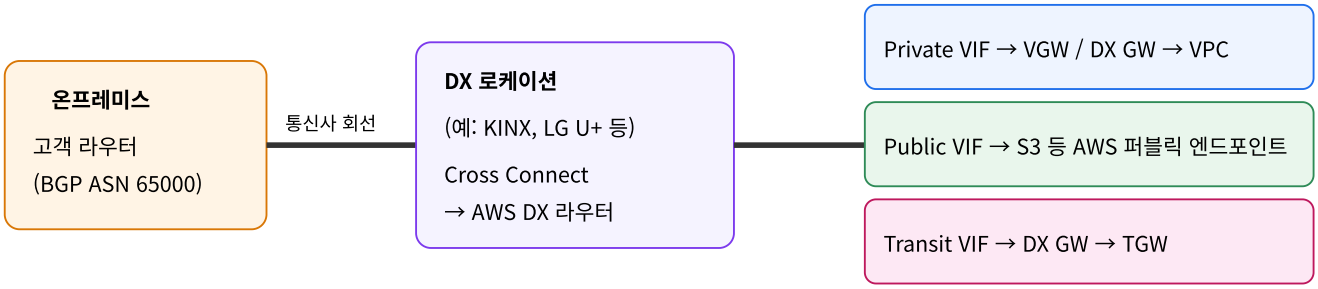


그림 5-2. Direct Connect 구성과 가상 인터페이스(VIF) 3종

개념	설명
Dedicated Connection	AWS와 직접 계약하는 물리 포트: 1, 10, 100, 400 Gbps
Hosted Connection	파트너가 나눠 제공: 50 Mbps ~ 25 Gbps
VIF (Virtual Interface)	802.1Q VLAN + BGP 세션. Private / Public / Transit 3종
Direct Connect Gateway	글로벌 리소스. 하나의 DX로 **여러 리전의 VPC(VGW) 또는 TGW**에 연결
LAG	같은 속도의 여러 연결을 묶음(LACP)
암호화	DX는 기본 암호화가 아님 → **MACsec**(10/100/400G 전용 연결) 또는 DX 위에 IPsec VPN

DX 고가용성 설계 [ANS]

- **최대 복원력**: 2개 DX 로케이션 × 각 2개 연결(장비 분리).
- **높은 복원력**: 2개 DX 로케이션 × 각 1개 연결.
- **비용 효율적 백업**: DX 1개 + Site-to-Site VPN 백업. AWS는 같은 프리픽스라면 **DX 경로를 VPN보다 우선**합니다.

BGP 경로 선택 제어

목적	방법
AWS → 온프레미스 방향 선택(인바운드 제어)	더 구체적인 프리픽스 광고, **AS_PATH Prepend**(경로를 길게 보여 덜 선호되게)
온프레미스 → AWS 방향 선택(아웃바운드 제어)	고객 라우터에서 **Local Preference** 조정
AWS 측 지역 선호도	BGP Community 태그(7224:7100 낮음 ~ 7224:7300 높음)

AWS의 경로 우선순위(VPC 라우트 테이블): **가장 긴 프리픽스** → 정적 경로 → 전파된 경로 중 DX BGP → 정적 VPN → BGP VPN 순.

5.6 하이브리드 DNS: Route 53 Resolver

구성	방향	용도
인바운드 엔드포인트	온프레미스 → AWS	온프레미스 DNS 서버가 AWS 프라이빗 호스팅 영역 이름을 조회 (조건부 포워딩 대상으로 엔드포인트 IP 지정)
아웃바운드 엔드포인트 + 전달 규칙	AWS → 온프레미스	`corp.example.local` 같은 사내 도메인 조회를 온프레미스 DNS로 전달
Resolver DNS Firewall	VPC 내부	악성 도메인 차단, DNS 기반 데이터 유출 방지

전달 규칙은 **RAM(Resource Access Manager)**으로 여러 계정에 공유할 수 있어, 중앙 네트워크 계정에서 한 번만 구성하면 됩니다.

5.7 VPC Flow Logs

ENI 단위로 오가는 IP 트래픽의 메타데이터(패킷 내용 아님)를 CloudWatch Logs, S3, Firehose로 기록합니다. VPC/서브넷/ENI 수준에서 켤 수 있습니다.

```
# 기본 형식 (version account-id interface-id srcaddr dstaddr srcport dstport protocol packets bytes start end action log-status)
2 123456789012 eni-0a1b 10.0.11.25 10.0.21.8 49152 3306 6 10 840 1700000000 1700000060 ACCEPT OK
2 123456789012 eni-0a1b 203.0.113.9 10.0.1.10 51515 22 6 1 40 1700000000 1700000060 REJECT OK
```

- `REJECT` 는 SG 또는 NACL이 막았다는 뜻. 인바운드는 ACCEPT인데 응답이 REJECT면 → **NACL 아웃바운드(임시 포트)**를 의심.
- 기록되지 않는 트래픽: Amazon DNS 질의, 메타데이터(169.254.169.254), DHCP, Windows 라이선스 활성화, 기본 VPC 라우터 예약 IP 트래픽 등.
- 사용자 지정 형식으로 `pkt-srcaddr`, `tcp-flags`, `flow-direction`, `traffic-path` 등 추가 가능 → NAT 뒤 실제 출발지 파악에 유용.
- 분석은 CloudWatch Logs Insights 또는 S3 + Athena.

패킷 내용까지 봐야 한다면 **Traffic Mirroring**(ENI 트래픽을 IDS/분석 장비로 복사)을 사용합니다. 경로 문제를 정적으로 분석하려면 **Reachability Analyzer**(A→B 도달 가능 여부와 막는 지점 표시), 의도치 않은 외부 노출은 **Network Access Analyzer**를 씁니다.

5.8 IPv6 in VPC

- VPC에 Amazon 제공 **/56** 또는 BYOIP/IPAM의 IPv6 CIDR 할당, 서브넷은 **/64**.
- IPv6 주소는 모두 **글로벌 유니캐스트(공인)**입니다. NAT가 없으므로, 아웃바운드만 허용하려면 **Egress-only Internet Gateway**를 사용(IPv6용 NAT GW 역할).

- IPv6 전용 서브넷 + **DNS64/NAT64**(NAT GW가 지원)로 IPv4 전용 서비스와 통신 가능.
- 퍼블릭 IPv4 과금 이후 IPv6 도입이 비용 절감 수단으로 주목받고 있습니다. ALB/NLB는 듀얼 스택 지원.

5.9 기타 네트워크 서비스

서비스	설명
AWS Network Firewall	관리형 상태 저장 방화벽(Suricata 규칙 호환). 도메인 필터링, IPS. 중앙 검사 VPC + TGW 구조로 많이 배포
Gateway Load Balancer	서드파티 방화벽 어플라이언스(Palo Alto, Fortinet 등)를 투명하게 끼워 넣음(GENEVE, 포트 6081)
RAM	서브넷, TGW, Resolver 규칙, 프리픽스 리스트 등을 다른 계정과 공유 → **VPC 공유**(공유 서브넷)로 중앙에서 네트워크 관리
Managed Prefix List	CIDR 묶음을 이름으로 관리해 SG·라우트에서 재사용
Elastic Fabric Adapter / 배치 그룹	HPC용 초저지연 네트워크(6장 참고)

연결 방식 선택 가이드

상황	추천
VPC 2~3개, 단순 연결	VPC Peering
VPC 수십 개 + 온프레미스, 세그먼트 분리	Transit Gateway (+ 멀티 리전이면 TGW Peering 또는 Cloud WAN)
특정 서비스만 노출, CIDR 겹침, SaaS 제공	PrivateLink / VPC Lattice
온프레미스 연결, 빠른 구축·저비용	Site-to-Site VPN
대용량·안정적 대역폭·저지연 하이브리드	Direct Connect (+ VPN 백업)

> 📝 **확인 문제 5**
>
> 1) VPC A–B, B–C가 피어링되어 있습니다. A에서 C로 통신하려면 어떻게 해야 하나요?
> 2) 프라이빗 서브넷 EC2가 S3에서 매일 5 TB를 읽어 NAT 비용이 큽니다. 가장 간단한 해결책은?
> 3) 온프레미스 서버가 AWS 프라이빗 호스팅 영역 `db.internal.example` 을 조회해야 합니다. 무엇이 필요한가요?
> 4) DX와 VPN이 같은 프리픽스를 광고합니다. AWS는 어느 쪽을 선호하나요?

CHAPTER 6

컴퓨팅

학습 목표 · EC2 인스턴스 유형 이름을 해석한다 · 구매 옵션별 비용·특징을 비교한다 · EBS 볼륨 유형과 인스턴스 스토어를 구분한다 · Auto Scaling 정책을 설계한다 · EC2 / 컨테이너 / 서버리스 중 알맞은 것을 고른다

6.1 EC2 인스턴스 유형

인스턴스 이름 `m7g.2xlarge` 해석: **m**(패밀리: 범용) **7**(세대) **g**(속성: Graviton ARM) **.2xlarge**(크기: 8 vCPU, 32 GiB).

패밀리	용도	특징 / 예시
T	버스트 범용	CPU 크레딧 기반. 평소 낮고 가끔 튀는 워크로드(개발 서버). unlimited 모드 주의
M	범용	vCPU:메모리 = 1:4. 웹/앱 서버 기본 선택
C	컴퓨팅 최적화	1:2. 배치, 인코딩, 고성능 웹
R / X / z	메모리 최적화	1:8 이상. 인메모리 DB, 캐시, 대규모 분석
I / D / Im	스토리지 최적화	로컬 NVMe 고IOPS. NoSQL, 데이터 웨어하우스
P / G / Trn / Inf	가속 컴퓨팅	GPU(ML 학습·추론, 그래픽), Trainium·Inferentia(AWS 자체 ML 칩)

속성 접미사: `g` =Graviton, `a` =AMD, `i` =Intel, `d` =로컬 NVMe, `n` =네트워크 강화, `e` =추가 메모리/스토리지, `flex` =저비용 유연형. **Graviton**은 같은 성능 대비 x86보다 보통 20% 안팎 저렴해 우선 검토할 가치가 있습니다.

Nitro 시스템과 네트워크 성능

최신 인스턴스는 **Nitro**(네트워크·스토리지·보안을 전용 하드웨어로 오프로드한 하이퍼바이저)에서 동작합니다. 네트워크 성능은 인스턴스 크기에 비례하며, "Up to 12.5 Gbps"처럼 표시된 것은 **버스트 성능**(기준 대역폭은 더 낮음)입니다. **ENA**(Enhanced Networking)는 최신 AMI에서 기본 활성화됩니다.

- 단일 흐름(5-tuple) 대역폭은 같은 배치 그룹 밖에서는 **5 Gbps**, 클러스터 배치 그룹 안에서는 10 Gbps(일부 최대 25 Gbps)로 제한됩니다.
- MTU: VPC 내부 9001(점보 프레임), IGW·VPN·피어링(리전 간) 경유 시 1500(VPN은 더 작음). Path MTU Discovery를 위해 ICMP Type 3 Code 4 허용 고려.

6.2 구매 옵션 [SAA 빈출]

옵션	할인율(대략)	특징	적합한 워크로드
온디맨드	기준	약정 없음, 초 단위 과금(Linux)	단기, 예측 불가
Savings Plans	최대 ~72%	1/3년 시간당 사용 금액 약정. Compute SP는 패밀리·리전·OS·Fargate·Lambda까지 유연	꾸준한 기본 사용량
예약 인스턴스(RI)	최대 ~72%	특정 인스턴스 속성 약정. Standard / Convertible. 용량 예약은 영역(Zonal) RI만	변경 없는 안정 워크로드
스팟 인스턴스	최대 ~90%	남는 용량. **2분 전 중단 알림** 후 회수될 수 있음	배치, CI, 상태 비저장 워커, 빅데이터
On-Demand 용량 예약	할인 없음	특정 AZ의 용량 확보(SP와 결합 시 할인)	이벤트·DR 용량 보장
전용 호스트 / 전용 인스턴스	비쌈	물리 서버 단독 사용. 전용 호스트는 소켓/코어 가시성 → BYOL 라이선스	규제, 라이선스 요건

> 💡 **비용 최적 조합**
>
> 기본 부하는 Savings Plans, 피크는 온디맨드, 중단 허용 작업은 스팟. Auto Scaling 그룹의 **혼합 인스턴스 정책**으로 온디맨드 기본 용량 + 스팟 비율을 지정하고, 여러 인스턴스 유형을 섞으면 스팟 회수 위험이 줄어듭니다 (`price-capacity-optimized` 할당 전략 권장).

6.3 AMI와 부트스트랩

- **AMI**: 루트 볼륨 스냅샷 + 메타데이터. 리전 단위 → 다른 리전에서 쓰려면 복사.
- **골든 AMI**: 공통 에이전트·보안 설정을 미리 굽기(EC2 Image Builder, Packer). 부팅 시간 단축.
- **User Data**: 최초 부팅 시 root로 실행되는 스크립트(cloud-init). 최대 16 KB.
- **Launch Template**: AMI, 유형, SG, 역할, User Data 등을 버전 관리. Auto Scaling에 필수(Launch Configuration은 지원 종료).

```
#!/bin/bash
dnf -y install nginx
echo "hello from $(hostname -f)" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

6.4 EC2 스토리지: EBS와 인스턴스 스토어

유형	분류	성능	용도
gp3	범용 SSD	기본 3,000 IOPS / 125 MB/s, 크기와 무관하게 최대 80,000 IOPS · 2,000 MB/s까지 별도 프로비저닝	**기본 선택**(gp2보다 저렴)
gp2	범용 SSD(구형)	GB당 3 IOPS, 버스트 크레딧	레거시 → gp3로 전환 권장
io2 Block Express	프로비저닝 IOPS SSD	최대 256,000 IOPS, 99.999% 내구성, Multi-Attach	대형 DB, 지연 민감
st1	처리량 최적 HDD	최대 500 MB/s	빅데이터, 로그(부팅 불가)
sc1	콜드 HDD	최대 250 MB/s, 최저가	드물게 읽는 데이터(부팅 불가)

- EBS는 **네트워크 블록 스토리지**이며 **같은 AZ**의 인스턴스에만 붙습니다. 다른 AZ로 옮기려면 스냅샷 → 복원.
- **스냅샷**: S3에 증분 저장, 리전 간 복사 가능. Data Lifecycle Manager 또는 AWS Backup으로 자동화. 휴지통(Recycle Bin)으로 실수 삭제 복구.
- **암호화**: KMS 기반, 계정 수준 "기본 암호화" 켜두기 권장. 미암호화 볼륨 → 스냅샷 → 암호화 복사 → 복원으로 전환.
- **DeleteOnTermination**: 루트 볼륨은 기본 true, 추가 볼륨은 기본 false.
- **인스턴스 스토어**: 호스트에 물리적으로 붙은 NVMe. 매우 빠르지만 **정지/종료 시 데이터 소실**(재부팅은 유지). 캐시·임시 데이터용.

6.5 배치 그룹

전략	배치	용도
Cluster	하나의 AZ, 같은 랙 근처	HPC, 초저지연·고대역폭 (장애 위험 집중)
Spread	서로 다른 하드웨어 랙, AZ당 최대 7개	소수의 중요 인스턴스 분산(예: 마스터 노드)
Partition	파티션(랙 그룹) 단위 분리, AZ당 최대 7 파티션	HDFS, Cassandra, Kafka 같은 토폴로지 인식 분산 시스템

6.6 Auto Scaling

Auto Scaling 그룹(ASG)은 **최소/원하는/최대 용량** 사이에서 인스턴스 수를 유지·조정하고, 비정상 인스턴스를 자동 교체합니다. 여러 AZ의 서브넷을 지정하면 AZ 간 균형을 맞춥니다.

조정 정책	설명
대상 추적 (권장)	"평균 CPU 50% 유지", "대상당 ALB 요청 1000" 같은 목표값 유지. 온도조절기처럼 동작
단계 조정	CloudWatch 경보 구간별로 +1, +3 등 다르게 조정
예약 조정	매일 09:00 최소 10대처럼 시간 기반
예측 조정	과거 패턴을 ML로 학습해 미리 확장

- **헬스 체크 유형**: 기본은 EC2 상태 검사. ALB 뒤에 있다면 **ELB 헬스 체크**를 켜야 앱 장애도 교체됩니다.
- **Warm-up / 쿨다운**: 새 인스턴스가 지표에 반영되기 전 과도한 확장 방지.
- **수명 주기 후크**: 시작/종료 직전에 대기(로그 수집, 등록 작업).
- **Instance Refresh**: 새 Launch Template 버전으로 롤링 교체. **Warm Pool**: 미리 초기화된 인스턴스 대기.

6.7 컨테이너와 서버리스

서비스	설명	선택 포인트
ECS	AWS 자체 컨테이너 오케스트레이터. 태스크 정의 → 서비스	AWS 통합이 단순, 러닝 커브 낮음
EKS	관리형 Kubernetes 컨트롤 플레인	K8s 생태계·이식성 필요 시. 운영 복잡도 높음
Fargate	ECS/EKS용 서버리스 컨테이너 실행 환경(노드 관리 없음)	서버 관리 부담 제거, 작업당 과금
ECR	컨테이너 이미지 레지스트리, 취약점 스캔	—
Lambda	이벤트 기반 함수. 최대 15분, 메모리 128 MB~10 GB(메모리에 비례해 CPU)	이벤트 처리, API 백엔드, 자동화. 콜드 스타트 고려
App Runner / Beanstalk	코드/이미지만 주면 배포·확장·LB를 자동 구성	빠른 웹앱 배포

컨테이너 네트워킹 핵심

- **ECS `awsvpc` 모드**(Fargate 필수): 태스크마다 ENI와 VPC IP, 태스크 단위 보안 그룹.
- **EKS VPC CNI**: 파드마다 VPC IP 할당 → IP가 빠르게 고갈됨. 대책: 큰 서브넷, `100.64.0.0/10` 보조 CIDR + Custom Networking, Prefix Delegation(ENI 슬롯당 /28 할당), IPv6 클러스터.
- **Lambda in VPC**: Hyperplane ENI를 통해 VPC 리소스 접근. VPC 안의 Lambda가 인터넷에 나가려면 **프라이빗 서브넷 + NAT GW** 필요(퍼블릭 서브넷에 둬도 퍼블릭 IP가 없어 불가).

> 📝 **확인 문제 6**
>
> 1) 야간 배치(중단되면 재시작 가능)를 가장 저렴하게 돌리려면?
> 2) EBS 볼륨을 AZ-a의 인스턴스에서 AZ-c의 인스턴스로 옮기려면?
> 3) ASG가 앱이 500 에러를 내는 인스턴스를 교체하지 않습니다. 무엇을 확인해야 하나요?
> 4) VPC 안의 Lambda가 외부 API 호출 시 타임아웃이 납니다. 원인과 해결책은?

CHAPTER 7

로드 밸런싱 (ELB)

학습 목표 · ALB / NLB / GWLB의 계층과 용도를 구분한다 · 리스너·규칙·대상 그룹의 관계를 이해한다 · 헬스 체크, 교차 영역 로드 밸런싱, 클라이언트 IP 보존을 설정한다

7.1 종류 비교

항목	ALB	NLB	GWLB
계층	L7 (HTTP/HTTPS/gRPC/WebSocket)	L4 (TCP/UDP/TLS/QUIC)	L3 (IP 패킷, GENEVE)
라우팅	경로, 호스트, 헤더, 쿼리, 메서드, 소스 IP	포트 기반	방화벽 어플라이언스로 투명 전달
IP	동적(DNS 이름으로 접근)	**AZ당 고정 IP / EIP 지정 가능**	—
지연·성능	ms 단위	초저지연, 초당 수백만 요청	—
대상	인스턴스, IP, Lambda	인스턴스, IP, ALB	인스턴스, IP
보안 그룹	지원	지원(생성 시 설정)	미지원
대표 용도	웹/마이크로서비스, 인증(OIDC/Cognito), WAF 연동	게임, IoT, 고정 IP 필요, PrivateLink	보안 장비 삽입

Classic Load Balancer(CLB)는 레거시이므로 신규 구축에는 사용하지 않습니다.

7.2 구성 요소

```
ALB (internet-facing, 퍼블릭 서브넷 2개 이상 AZ)
  └── 리스너 :80   → 규칙: HTTPS로 301 리다이렉트
  └── 리스너 :443  (ACM 인증서, 보안 정책 ELBSecurityPolicy-TLS13-1-2-2021-06)
       ├── 규칙 1: Host = api.example.com          → TG-api
       ├── 규칙 2: Path = /images/*                → TG-static
       └── 기본 규칙                                → TG-web
대상 그룹 TG-web: 프로토콜 HTTP:8080, 헬스 체크 GET /health (200), 대상 = ASG 인스턴스
```

7.3 주요 설정

설정	설명
헬스 체크	경로, 간격(기본 30초), 정상/비정상 임계값, 성공 코드. **헬스 체크 엔드포인트는 DB 등 의존성까지 과도하게 확인하지 않도록**(연쇄 장애 방지)
교차 영역(Cross-Zone) LB	모든 AZ의 대상에 고르게 분산. ALB 기본 켜짐(무료), NLB/GWLB 기본 꺼짐(켜면 AZ 간 전송 요금)
등록 취소 지연(Draining)	대상 제거 시 진행 중 요청 처리 시간(기본 300초)
고정 세션(Sticky)	쿠키로 같은 대상 유지. 가능하면 세션을 ElastiCache/DynamoDB로 외부화하는 것이 정석
클라이언트 IP	ALB: `X-Forwarded-For` 헤더. NLB: 인스턴스 대상은 원본 IP 보존, 그 외 **Proxy Protocol v2**
유휴 제한 시간	ALB 기본 60초. 긴 요청은 늘리거나 비동기로
Access Logs	S3에 요청 로그 저장 → 504/502 분석의 1차 자료

> **⚠ 내부(Internal) vs 인터넷 연결(Internet-facing)**
> 스킴은 생성 후 변경 불가. 인터넷 연결 ALB는 **퍼블릭 서브넷**에, 대상은 **프라이빗 서브넷**에 둡니다. 대상 SG는 ALB의 SG에서 오는 트래픽만 허용하세요. 각 서브넷은 최소 /27 이상, 여유 IP 8개 이상이 필요합니다.

CHAPTER 8

스토리지

학습 목표 · 블록/파일/객체 스토리지를 구분한다 · S3 스토리지 클래스, 수명 주기, 버전 관리, 복제, 암호화, 접근 제어를 설계한다 · EFS/FSx의 용도를 안다 · 대용량 데이터 이전 방법을 고른다

8.1 스토리지 유형

유형	AWS	접근 방식	특징
블록	EBS, 인스턴스 스토어	디스크로 마운트	단일 인스턴스(Multi-Attach 예외), 저지연
파일	EFS(NFS), FSx	여러 서버가 공유 마운트	공유 파일 시스템
객체	S3	HTTP API (PUT/GET)	무제한, 11 9s(99.999999999%) 내구성

8.2 Amazon S3

버킷(리전 단위, 이름 전 세계 고유) 안에 객체(키 + 값 + 메타데이터, 최대 5 TB)를 저장합니다. "폴더"는 키의 접두사(prefix)일 뿐입니다. 100 MB 이상은 **멀티파트 업로드**를 권장합니다. S3는 **강력한 쓰기 후 읽기 일관성**을 제공합니다.

스토리지 클래스

클래스	특징	용도
Standard	3개 이상 AZ, 밀리초 접근	자주 쓰는 데이터
Intelligent-Tiering	접근 패턴에 따라 자동 계층 이동(소액 모니터링 요금)	패턴을 모를 때 기본 선택
Standard-IA	저장 저렴, 조회 요금, 최소 30일	가끔 읽는 백업
One Zone-IA	단일 AZ → AZ 손실 시 데이터 손실 가능	재생성 가능한 데이터
Express One Zone	단일 AZ, 한 자릿수 ms 지연(디렉터리 버킷)	ML 학습, 고성능 분석
Glacier Instant Retrieval	ms 접근, 최소 90일	분기별 접근 아카이브
Glacier Flexible Retrieval	분~시간 복원, 최소 90일	백업 아카이브
Glacier Deep Archive	12~48시간 복원, 최저가, 최소 180일	규정상 장기 보관(7~10년)

핵심 기능

- **버전 관리**: 덮어쓰기·삭제로부터 보호(삭제 시 삭제 마커). 복제·Object Lock의 전제 조건.
- **수명 주기 규칙**: 30일 후 IA, 90일 후 Glacier, 365일 후 삭제, 이전 버전 정리, 미완료 멀티파트 업로드 삭제.
- **복제**: CRR(리전 간), SRR(같은 리전). 버전 관리 필요, 기존 객체는 Batch Replication.
- **Object Lock**: WORM. Compliance 모드는 루트도 삭제 불가 → 랜섬웨어 대비 백업.

- **암호화**: 모든 신규 객체는 기본 SSE-S3 암호화. 키 관리·감사가 필요하면 SSE-KMS(+ 버킷 키로 KMS 비용 절감), DSSE-KMS(이중 암호화), SSE-C(고객 키).
- **성능**: 접두사당 초당 3,500 PUT / 5,500 GET. 접두사를 나누면 선형 확장. 원거리 업로드는 Transfer Acceleration.
- **이벤트 알림**: 객체 생성 시 Lambda/SQS/SNS/EventBridge 호출.
- **Presigned URL**: 일정 시간만 유효한 임시 업로드/다운로드 URL.

S3 접근 제어 — 어떻게 막고 어떻게 열까

1. **퍼블릭 액세스 차단(Block Public Access)**: 계정·버킷 수준. 기본 켜짐. 정말 공개가 필요한 경우가 아니면 끄지 마세요.
2. **Object Ownership = Bucket owner enforced**: ACL 비활성화(기본값). 정책만으로 관리.
3. **버킷 정책**: 크로스 계정, VPC 엔드포인트 제한, TLS 강제(`aws:SecureTransport`).
4. **정적 웹 호스팅**은 버킷을 직접 공개하기보다 **CloudFront + OAC(Origin Access Control)**로 버킷은 비공개 유지.
5. **Access Points**: 대규모 공유 데이터셋에 팀별 접근 정책 분리.

8.3 파일 스토리지

서비스	설명
EFS	관리형 NFS v4. 여러 AZ에서 수천 대가 동시 마운트, 자동 확장. Linux 전용. 스토리지 클래스(Standard/IA/Archive), 처리량 모드(Elastic 권장). 마운트 타깃은 AZ마다, SG에서 **TCP 2049** 허용
FSx for Windows	SMB, Active Directory 통합, Windows 워크로드
FSx for Lustre	HPC·ML용 초고성능, S3와 연동
FSx for NetApp ONTAP	NFS/SMB/iSCSI 멀티 프로토콜, 스냅샷·복제 — 온프레미스 NetApp 이전
FSx for OpenZFS	ZFS 기반 NFS, 저지연

8.4 하이브리드 스토리지 & 데이터 이전

서비스	용도
Storage Gateway	온프레미스에서 S3/FSx를 로컬 스토리지처럼: S3 File Gateway(NFS/SMB → S3), FSx File Gateway, Volume Gateway(iSCSI), Tape Gateway(가상 테이프 → Glacier)
DataSync	NFS/SMB/HDFS/다른 클라우드 → S3/EFS/FSx 온라인 전송·동기화(검증·스케줄)
Transfer Family	SFTP/FTPS/FTP/AS2 → S3/EFS 관리형 엔드포인트
Snowball Edge	물리 장치로 TB~PB 오프라인 이전 (신규 고객 제공 정책이 바뀌고 있어 가용성 확인 필요)

💡 네트워크로 옮길까, 장비로 옮길까

대략 계산: 1 Gbps 회선을 80% 사용하면 하루 ≈ 8.6 TB. 100 TB면 약 12일. 기간 내 전송이 불가능하면 오프라인 장치, 가능하면 DataSync(+DX)가 일반적입니다.

📝 확인 문제 8

1) 로그를 30일간 자주 보고, 1년간 가끔 보고, 7년 보관 후 삭제해야 합니다. 수명 주기를 설계하세요.
2) 여러 AZ의 EC2 웹 서버가 업로드 파일을 공유 디렉터리로 써야 합니다. 무엇을 쓰나요?
3) S3 정적 사이트를 버킷 공개 없이 HTTPS로 서비스하려면?

CHAPTER 9

데이터베이스

학습 목표 · Multi-AZ와 읽기 전용 복제본의 차이를 정확히 안다 · Aurora의 구조적 장점을 설명한다 · DynamoDB의 키 설계와 용량 모드를 이해한다 · 워크로드에 맞는 DB를 고른다

9.1 Amazon RDS

관리형 관계형 DB: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2. AWS가 프로비저닝·패치·백업·장애 조치를 담당합니다(OS 접근 불가, RDS Custom 예외).

구분	Multi-AZ (인스턴스 배포)	Multi-AZ DB 클러스터	읽기 전용 복제본
목적	**고가용성**	고가용성 + 읽기	**읽기 확장**
복제	동기	반동기, 리더 2개	비동기
읽기 가능?	스탠바이는 읽기 불가	리더 2개 읽기 가능	가능(별도 엔드포인트)
장애 조치	자동(보통 60~120초), DNS 엔드포인트 동일	자동(보통 35초 이내)	수동 승격
범위	같은 리전	같은 리전	같은/다른 리전 (DR 용도 가능)

- **자동 백업**: 보존 기간(최대 35일) 내 **특정 시점 복구(PITR)**. 복구는 항상 **새 DB 인스턴스**로 생성됩니다.
- **수동 스냅샷**은 삭제 전까지 보존. 다른 리전/계정에 복사 가능.
- **RDS Proxy**: 커넥션 풀링으로 Lambda의 대량 연결 문제 해결, 장애 조치 시간 단축, IAM 인증.
- **DB 서브넷 그룹**: 최소 2개 AZ의 프라이빗 서브넷. 퍼블릭 액세스는 "아니요".
- 암호화는 생성 시에만 설정 가능 → 미암호화 DB는 스냅샷 → 암호화 복사 → 복원.

9.2 Amazon Aurora

MySQL/PostgreSQL 호환 클라우드 네이티브 DB. 컴퓨팅과 스토리지가 분리되어 있고, 스토리지는 **3개 AZ에 6개 사본**으로 자동 복제(쓰기 4/6, 읽기 3/6 쿼럼)됩니다.

- 복제본 최대 15개, 복제 지연 대개 수십 ms. **라이터 엔드포인트 / 리더 엔드포인트(부하 분산) / 사용자 지정 엔드포인트**.
- 스토리지 자동 확장(최대 128 TiB 이상, 엔진 버전에 따라 상이).
- **Aurora Serverless v2**: ACU 단위로 초 단위 자동 확장(0 ACU까지 일시 중지 지원).
- **Global Database**: 다른 리전에 1초 미만 지연 복제, 리전 장애 시 1분 내외 승격 → 리전 DR.
- **Backtrack**(MySQL): 스냅샷 복원 없이 시점 되돌리기. **클론**: 카피 온 라이트로 빠른 복제.
- **Aurora DSQL**: 분산 SQL, 멀티 리전 액티브-액티브(2025 정식 출시).

9.3 Amazon DynamoDB

완전 관리형 키-값/문서 NoSQL. 한 자릿수 ms 지연을 어떤 규모에서도 유지합니다. 서버 개념이 없습니다.

개념	설명
기본 키	파티션 키(PK) 단독 또는 PK + 정렬 키(SK). **PK는 카디널리티가 높게**(핫 파티션 방지)
항목 크기	최대 400 KB
보조 인덱스	GSI(다른 PK/SK, 언제든 추가), LSI(같은 PK 다른 SK, 테이블 생성 시에만)
용량 모드	온디맨드(요청당 과금, 예측 불가 트래픽) / 프로비저닝(RCU·WCU + Auto Scaling, 예측 가능 트래픽에 저렴)
일관성	기본 최종 일관 읽기, 강력한 일관 읽기 선택 가능(2배 RCU)
부가 기능	Streams(변경 이벤트 → Lambda), TTL(자동 만료), PITR, Global Tables(멀티 리전 액티브-액티브), 트랜잭션, DAX(마이크로초 캐시)

> 💡 **DynamoDB 설계의 핵심**
> RDB처럼 "엔터티별 테이블 + 조인"이 아니라 **접근 패턴을 먼저 정의**하고 그에 맞게 키를 설계합니다(Single-table design). Scan은 비싸고 느리므로 Query로 해결되도록 설계하세요.

9.4 ElastiCache와 기타 DB

서비스	용도
ElastiCache (Valkey/Redis OSS, Memcached)	인메모리 캐시, 세션 저장소, 순위표, Pub/Sub. Valkey/Redis는 복제·Multi-AZ·백업 지원, Memcached는 단순 멀티스레드 캐시. Serverless 옵션
MemoryDB	내구성 있는 Redis 호환 주 DB(트랜잭션 로그 Multi-AZ)
DocumentDB	MongoDB 호환 문서 DB
Neptune	그래프 DB (소셜, 추천, 사기 탐지)
Keyspaces	Cassandra 호환
Timestream	시계열 (IoT, 지표)
Redshift	데이터 웨어하우스(OLAP), 컬럼형
OpenSearch Service	검색, 로그 분석

캐싱 전략

- **Lazy Loading(Cache-aside)**: 캐시 미스 시 DB 조회 후 캐시에 저장. 오래된 데이터 가능 → TTL 설정.
- **Write-through**: DB 쓰기 시 캐시도 갱신. 항상 최신이지만 안 쓰는 데이터도 캐시됨.

📝 확인 문제 9

1) 보고서 쿼리로 운영 DB가 느려집니다. Multi-AZ를 켜면 해결될까요?

2) 서울 리전 장애 시 도쿄에서 1분 내 DB 쓰기를 재개해야 합니다. 무엇을 쓰나요?

3) 수천 개의 Lambda 동시 실행이 RDS 연결 수 한도를 초과합니다. 해결책은?

CHAPTER 10

DNS와 콘텐츠 전송

학습 목표 · Route 53 라우팅 정책 7가지와 헬스 체크를 활용한다 · CloudFront의 캐시 동작과 오리진 보호를 설정한다 · CloudFront와 Global Accelerator를 구분한다

10.1 Route 53

고가용성 DNS 서비스(SLA 100%), 도메인 등록, 헬스 체크를 제공합니다.

- **퍼블릭 호스팅 영역**: 인터넷에서 조회. **프라이빗 호스팅 영역**: 연결된 VPC 안에서만 조회(Split-horizon DNS 가능).
- 도메인을 다른 등록 기관에서 샀다면, 그곳의 NS 레코드를 Route 53 호스팅 영역의 4개 네임서버로 바꿔야 합니다.
- **TTL**: 변경 전에 미리 TTL을 낮춰 두면 전환이 빨라집니다. Alias 레코드는 대상 리소스의 TTL을 따릅니다.

라우팅 정책 [SAA 빈출]

정책	동작	용도
단순(Simple)	하나의 레코드(여러 IP면 무작위 순서). 헬스 체크 불가	단일 리소스
가중치(Weighted)	비율로 분배(예: 90:10)	카나리 배포, 블루/그린
지연 시간(Latency)	사용자에게 지연이 가장 낮은 AWS 리전	멀티 리전 성능
장애 조치(Failover)	Primary 비정상 시 Secondary	액티브-패시브 DR
지리적 위치 (Geolocation)	사용자 위치(국가·대륙) 기준	현지화, 콘텐츠 라이선스 제한
지리 근접성 (Geoproximity)	거리 + 바이어스로 영역 크기 조정(Traffic Flow)	점진적 트래픽 이동
IP 기반	클라이언트 CIDR 기준	ISP별 최적 엔드포인트
다중 값 응답	정상 레코드 최대 8개 반환	간단한 클라이언트 측 분산(LB 대체 아님)

헬스 체크: 엔드포인트(HTTP/HTTPS/TCP), 다른 헬스 체크 조합(계산된), CloudWatch 경보 기반. 헬스 체커는 인터넷에서 접근하므로 프라이빗 리소스는 CloudWatch 경보 방식을 사용합니다. 방화벽에서 Route 53 헬스 체커 IP 대역을 허용해야 합니다.

10.2 CloudFront

전 세계 수백 개 엣지 로케이션에서 콘텐츠를 캐시하는 CDN입니다. 정적 파일뿐 아니라 동적 콘텐츠(API)도 AWS 백본을 통한 최적 경로·TLS 종료·연결 재사용으로 빨라집니다.

개념	설명
오리진	S3, ALB, EC2, API Gateway, 임의 HTTP 서버, 그리고 VPC 오리진(프라이빗 서브넷의 ALB/NLB/EC2를 퍼블릭 노출 없이)
캐시 동작(Behavior)	경로 패턴별 오리진·캐시 정책 지정 (`/api/*` 는 캐시 안 함, `/static/*` 는 1일 캐시)
캐시 정책 / 오리진 요청 정책	캐시 키(헤더·쿠키·쿼리) vs 오리진으로 전달할 값 분리. 캐시 키가 많을수록 적중률↓
OAC	S3 오리진을 CloudFront만 접근 가능하게(버킷 정책에서 CloudFront 서비스 주체 + 배포 ARN 조건)
ALB 오리진 보호	ALB SG에 CloudFront 관리형 프리픽스 리스트만 허용 + 커스텀 헤더 검증, 또는 VPC 오리진
무효화(Invalidation)	캐시 강제 삭제(비용 발생) → 파일명 버전 관리(`app.3f2a.js`)가 더 좋음
서명된 URL/쿠키	유료 콘텐츠 접근 제한
엣지 컴퓨팅	CloudFront Functions(경량 JS, 헤더 조작·리다이렉트), Lambda@Edge(무거운 로직)
보안	WAF 연결, Shield Standard 기본, 지리적 제한, TLS 인증서는 **us-east-1의 ACM**에서 발급

10.3 Global Accelerator

2개의 **고정 애니캐스트 IP**를 제공하고, 사용자를 가장 가까운 엣지에서 AWS 백본으로 태워 리전의 엔드포인트(ALB, NLB, EC2, EIP)로 보냅니다.

비교	CloudFront	Global Accelerator
캐시	있음	없음(프록시)
프로토콜	HTTP/HTTPS/WebSocket	TCP/UDP 전반
IP	변동	고정 애니캐스트 IP 2개
장애 조치	오리진 그룹 장애 조치	리전 간 수십 초 내 장애 조치(DNS 캐시 영향 없음)
용도	웹 콘텐츠·API 가속	게임, IoT, VoIP, 고정 IP 화이트리스트 필요 시

CHAPTER 11

보안 서비스

학습 목표 · 데이터 보호(KMS, Secrets Manager, ACM)를 설정한다 · 경계 보호(WAF, Shield, Network Firewall)를 구분한다 · 위협 탐지·컴플라이언스 서비스의 역할을 안다

11.1 데이터 보호

서비스	설명
KMS	암호화 키 관리. AWS 관리형 키 / **고객 관리형 키(CMK)**(키 정책·교체·감사 제어). **봉투 암호화**: 데이터는 데이터 키로, 데이터 키는 KMS 키로 암호화. 키 정책이 1차 권한 원천. 멀티 리전 키 지원
CloudHSM	전용 HSM(FIPS 140-3 Level 3), 키를 고객이 단독 통제
Secrets Manager	DB 비밀번호·API 키 저장, **자동 교체**(RDS 연동), 크로스 계정 공유
SSM Parameter Store	설정값·비밀(SecureString) 저장. 표준 계층 무료, 자동 교체 없음
ACM	무료 공인 TLS 인증서 자동 갱신(ELB, CloudFront, API Gateway). EC2에 직접 설치는 불가(내보내기 가능한 인증서 옵션은 유료). 사설 CA도 제공
Macie	S3의 개인정보(PII) 자동 탐지

ℹ 전송 중 / 저장 중 암호화

저장 중(at rest): EBS·S3·RDS·DynamoDB 모두 KMS로 간단히 활성화. 전송 중(in transit): TLS 사용, S3 버킷 정책으로 HTTPS 강제, RDS는 `require_secure_transport` / `rds.force_ssl` , 같은 VPC 내 최신 Nitro 인스턴스 간 트래픽은 하드웨어 수준에서 자동 암호화.

11.2 경계(네트워크/애플리케이션) 보호

서비스	설명
AWS WAF	L7 방화벽. CloudFront, ALB, API Gateway, AppSync, Cognito에 연결. SQLi/XSS 관리형 규칙, 속도 기반 규칙(IP당 요청 제한), 지역 차단, Bot Control
Shield Standard	모든 고객 무료, L3/L4 DDoS 방어
Shield Advanced	유료(월 정액). L7 DDoS 대응, 24/7 SRT 지원, DDoS로 인한 확장 비용 보호
Network Firewall	VPC 수준 상태 저장 방화벽/IPS, 도메인 필터링(5장)
Firewall Manager	Organizations 전체에 WAF·SG·Network Firewall 정책 일괄 적용
Verified Access	VPN 없이 사내 앱에 제로 트러스트 접근

11.3 탐지와 대응

서비스	설명
GuardDuty	CloudTrail, VPC Flow Logs, DNS 로그 등을 ML로 분석해 위협 탐지(채굴, 자격 증명 탈취, 이상 API 호출). 에이전트 불필요, 켜기만 하면 됨
Inspector	EC2·ECR 이미지·Lambda의 취약점(CVE)과 네트워크 노출 스캔
Security Hub	보안 결과 통합 대시보드, CIS/AWS 모범 사례 표준 점검
Detective	보안 사건의 근본 원인 조사(그래프 분석)
Config	리소스 구성 변경 이력과 규칙 준수 평가(12장)
Audit Manager / Artifact	감사 증적 수집 / AWS 컴플라이언스 보고서(SOC, ISO) 다운로드

> 💡 **신규 계정 보안 기본 세트**
>
> 루트 MFA → CloudTrail(조직 트레일) → GuardDuty → Security Hub → Config → IAM Access Analyzer → S3 퍼블릭 액세스 차단(계정 수준) → EBS 기본 암호화 → IMDSv2 기본값 → 예산 알림. Control Tower를 쓰면 상당 부분이 자동 구성됩니다.

CHAPTER 12

모니터링·감사·운영

학습 목표 · CloudWatch 지표·로그·경보를 구성한다 · CloudTrail과 Config의 차이를 안다 · Systems Manager로 SSH 없이 서버를 관리한다

12.1 CloudWatch

기능	설명
Metrics	EC2 기본 지표 5분(세부 모니터링 1분). **메모리·디스크 사용률은 기본 지표에 없음** → CloudWatch Agent 설치 필요. 사용자 지정 지표(고해상도 1초)
Alarms	지표 임계값 → SNS 알림, Auto Scaling, EC2 작업(재부팅/복구). 복합 경보로 노이즈 감소. 이상 탐지 밴드
Logs	로그 그룹/스트림, 보존 기간 설정(기본 무기한 → 비용 주의), Logs Insights 쿼리, 지표 필터, 구독 필터(→ Lambda/Firehose)
Dashboards	교차 계정·교차 리전 대시보드
Synthetics / RUM	카나리 스크립트로 외부 모니터링 / 실사용자 모니터링
Application Signals / X-Ray	APM, 분산 추적(서비스 맵, 지연 구간 분석). OpenTelemetry 기반 수집 권장
Container Insights	ECS/EKS 지표·로그

```
# Logs Insights: 최근 1시간 5xx가 많은 경로 Top 10 (ALB 로그를 CloudWatch로 수집한 경우 예시)
fields @timestamp, request_url, elb_status_code
| filter elb_status_code >= 500
| stats count(*) as errors by request_url
| sort errors desc
| limit 10
```

12.2 CloudTrail vs Config

구분	CloudTrail	Config
질문	**"누가, 언제, 어떤 API를 호출했나?"**	**"리소스 구성이 어떻게 바뀌었고, 규칙을 준수하나?"**
데이터	API 호출 이벤트(관리 이벤트 기본, 데이터 이벤트는 별도 설정·과금)	리소스 구성 스냅샷·변경 타임라인
기본 보존	이벤트 기록 90일(콘솔) → 장기 보존은 트레일로 S3에	설정한 기간
예시	SG 규칙을 삭제한 사용자 찾기	"SSH 0.0.0.0/0 허용 SG가 있나" 규칙 → 자동 수정(SSM 문서)

조직 단위 **Organization Trail**을 만들고, 로그는 별도 Log Archive 계정의 S3(Object Lock, 로그 파일 무결성 검증)에 보관하는 것이 표준입니다. CloudTrail Lake로 SQL 쿼리도 가능합니다.

12.3 Systems Manager (SSM)

기능	설명
Session Manager	**SSH 포트·배스천·키 없이** 브라우저/CLI로 쉘 접속. IAM으로 권한 제어, 세션 로그를 S3/CloudWatch에 기록. 포트 포워딩으로 프라이빗 RDS 접근도 가능
Run Command	여러 서버에 명령 일괄 실행
Patch Manager	패치 기준선·유지 관리 기간으로 자동 패치
Parameter Store	설정·비밀 저장
Automation	런북 자동화(AMI 생성, 장애 복구 절차)
Inventory / Fleet Manager	설치 소프트웨어 목록, 서버 원격 관리

```
# 인스턴스 접속 (SSM Agent + 역할 AmazonSSMManagedInstanceCore 필요)
aws ssm start-session --target i-0123456789abcdef0
# 로컬 5432 → 프라이빗 RDS 5432 포트 포워딩
aws ssm start-session --target i-0123456789abcdef0 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters host=mydb.xxxx.ap-northeast-2.rds.amazonaws.com,portNumber=5432,localPortNumber=5432
```

그 밖에 **Trusted Advisor**(비용·보안·한도 점검), **Health Dashboard**(내 계정에 영향 있는 AWS 장애·예정 작업), **Service Quotas**(한도 조회·증가 요청), **Compute Optimizer**(Right-sizing 추천)를 정기적으로 확인하세요.

CHAPTER 13

애플리케이션 통합

학습 목표 · 동기 결합을 비동기 메시징으로 풀어 장애 전파를 막는다 · SQS / SNS / EventBridge / Kinesis의 차이를 안다

서비스	모델	특징
SQS	큐(Pull)	Standard(거의 무제한 처리량, 최소 1회 전달, 순서 비보장) / FIFO(순서·정확히 1회 처리). 가시성 제한 시간, DLQ, 롱 폴링, 메시지 최대 1 MiB(그 이상은 S3 + 확장 클라이언트). ASG를 큐 길이로 확장
SNS	Pub/Sub(Push)	하나의 메시지를 여러 구독자(SQS, Lambda, HTTP, 이메일, SMS)에게 **팬아웃**. 메시지 필터링
EventBridge	이벤트 버스	이벤트 패턴 기반 라우팅, AWS 서비스·SaaS 이벤트, 스케줄러, 아카이브/재생, 크로스 계정
Kinesis Data Streams	스트리밍	샤드 기반 실시간 스트림, 순서 보장(샤드 내), 재처리(보존 최대 365일). 대안: MSK(Kafka)
Data Firehose	전송	스트림을 S3/Redshift/OpenSearch로 거의 실시간 적재(버퍼링, 변환)
Step Functions	워크플로	상태 머신으로 여러 단계 오케스트레이션, 재시도·보상 트랜잭션
API Gateway	API 프런트	REST/HTTP/WebSocket API, 인증(Cognito/JWT/IAM/Lambda), 스로틀링, 사용량 계획. 엔드포인트 유형: 엣지 최적화·리전·**프라이빗**(VPC 엔드포인트로만 접근)

💡 SNS + SQS 팬아웃 패턴

주문 이벤트 → SNS 토픽 → (결제 큐, 배송 큐, 분석 큐) 각각 SQS 구독. 소비자가 독립적으로 장애·확장되고, 실패 메시지는 각 큐의 DLQ로 격리됩니다.

CHAPTER 14

Infrastructure as Code

학습 목표 · 콘솔 클릭 대신 코드로 인프라를 만들고 지운다 · CloudFormation, CDK, Terraform의 차이를 안다 · 상태 관리와 드리프트를 이해한다

14.1 왜 IaC인가

- **재현성**: dev/stg/prod를 같은 코드로. **리뷰 가능**: 변경을 PR로 검토. **복구**: 리전 DR 시 코드로 재구축. **정리**: 실습 후 한 번에 삭제.

도구	언어	상태 관리	특징
CloudFormation	YAML/JSON	AWS가 스택으로 관리	AWS 네이티브, 변경 세트, 드리프트 탐지, StackSets(다계정 배포)
AWS CDK	TypeScript, Python 등	CloudFormation으로 합성	프로그래밍 언어의 추상화(Construct)
Terraform / OpenTofu	HCL	state 파일(S3 백엔드 + 잠금)	멀티 클라우드, 거대한 모듈 생태계, plan으로 변경 미리보기

14.2 CloudFormation 예제: VPC + 퍼블릭 서브넷

```
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  VpcCidr: { Type: String, Default: 10.0.0.0/16 }
Resources:
  Vpc:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !Ref VpcCidr
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags: [{ Key: Name, Value: study-vpc }]
  Igw: { Type: AWS::EC2::InternetGateway }
  IgwAttach:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties: { VpcId: !Ref Vpc, InternetGatewayId: !Ref Igw }
  PublicSubnetA:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref Vpc
      CidrBlock: !Select [0, !Cidr [!Ref VpcCidr, 6, 8]]   # 10.0.0.0/24
      AvailabilityZone: !Select [0, !GetAZs ""]
      MapPublicIpOnLaunch: true
  PublicRt: { Type: AWS::EC2::RouteTable, Properties: { VpcId: !Ref Vpc } }
  DefaultRoute:
    Type: AWS::EC2::Route
    DependsOn: IgwAttach
    Properties: { RouteTableId: !Ref PublicRt, DestinationCidrBlock: 0.0.0.0/0, GatewayId: !Ref Igw }
  PublicAssocA:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties: { SubnetId: !Ref PublicSubnetA, RouteTableId: !Ref PublicRt }
Outputs:
  VpcId: { Value: !Ref Vpc, Export: { Name: study-vpc-id } }
```

```
aws cloudformation deploy --template-file vpc.yaml --stack-name study-vpc
aws cloudformation delete-stack --stack-name study-vpc
```

14.3 Terraform 예제: 3계층 VPC (모듈 없이 핵심만)

```
terraform {
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
  backend "s3" {                          # 팀 작업 시 원격 state
    bucket       = "my-tf-state-bucket"
    key          = "study/vpc.tfstate"
    region       = "ap-northeast-2"
    use_lockfile = true
  }
}
provider "aws" {
  region = "ap-northeast-2"
  default_tags { tags = { Project = "study" } }
}
locals {
  azs  = ["ap-northeast-2a", "ap-northeast-2c"]
  cidr = "10.0.0.0/16"
}
resource "aws_vpc" "main" {
  cidr_block = local.cidr
  enable_dns_hostnames = true
}
resource "aws_subnet" "public" {
  count                   = length(local.azs)
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(local.cidr, 8, count.index + 1)   # 10.0.1.0/24, 10.0.2.0/24
  availability_zone       = local.azs[count.index]
  map_public_ip_on_launch = true
}
resource "aws_subnet" "app" {
  count             = length(local.azs)
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(local.cidr, 8, count.index + 11)        # 10.0.11.0/24 …
  availability_zone = local.azs[count.index]
}
resource "aws_internet_gateway" "igw" { vpc_id = aws_vpc.main.id }
resource "aws_eip" "nat" {
  count  = length(local.azs)
  domain = "vpc"
}
resource "aws_nat_gateway" "nat" {
  count         = length(local.azs)               # AZ마다 NAT (고가용성)
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
}
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
resource "aws_route_table" "app" {
  count  = length(local.azs)
  vpc_id = aws_vpc.main.id
```

```
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat[count.index].id
  }
}
resource "aws_route_table_association" "public" {
  count = length(local.azs)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}
resource "aws_route_table_association" "app" {
  count = length(local.azs)
  subnet_id      = aws_subnet.app[count.index].id
  route_table_id = aws_route_table.app[count.index].id
}
resource "aws_vpc_endpoint" "s3" {                # NAT 비용 절감
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.ap-northeast-2.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids   = aws_route_table.app[*].id
}
```

```
terraform init && terraform plan -out tfplan && terraform apply tfplan
terraform destroy      # 실습 후 반드시 정리 (NAT GW는 시간당 과금!)
```

> **💡 실무 팁**
>
> 검증된 커뮤니티 모듈(`terraform-aws-modules/vpc/aws`)로 시작해도 좋지만, 처음에는 위처럼 리소스를 직접 작성해 보며 각 리소스의 관계를 익히는 것을 추천합니다. state 파일에는 비밀값이 들어갈 수 있으니 S3 암호화·접근 제한을 하세요.

CHAPTER 15

고가용성·재해 복구·비용 최적화

학습 목표 · RTO/RPO로 요구사항을 수치화한다 · DR 전략 4가지를 비용·복구 시간으로 비교한다 · 비용 구조(특히 데이터 전송)를 이해하고 줄인다

15.1 고가용성 설계 원칙

- **단일 장애점(SPOF) 제거**: 모든 계층을 최소 2개 AZ에. NAT GW도 AZ별로.
- **상태 비저장(Stateless) 앱**: 세션·파일을 인스턴스 밖(ElastiCache, DynamoDB, S3, EFS)으로 → 자유로운 확장·교체.
- **느슨한 결합**: 큐로 계층 분리 → 한 계층 장애가 전체로 번지지 않음.
- **자동 복구**: ASG + ELB 헬스 체크, RDS Multi-AZ, Route 53 장애 조치.
- **재시도 + 지수 백오프 + 지터**, 타임아웃, 서킷 브레이커.
- **정적 안정성(Static Stability)**: 장애 시 새 자원을 만들지 않아도 버틸 만큼 미리 확보(예: 3 AZ에 각 50% 용량 → 1 AZ 손실 시에도 100%).

15.2 RTO / RPO

RPO(Recovery Point Objective): 얼마만큼의 데이터 손실을 허용하는가(시간). **RTO(Recovery Time Objective)**: 서비스가 복구되기까지 허용 시간.

DR 전략	방식	RPO/RTO	비용
백업 & 복원	다른 리전에 백업(AWS Backup, 스냅샷 복사, S3 CRR). 장애 시 IaC로 재구축 후 복원	시간 / 수 시간~하루	$
파일럿 라이트	DB 등 핵심 데이터만 상시 복제, 앱 서버는 꺼두거나 AMI만 준비	분 / 수십 분	$$
웜 스탠바이	축소된 규모의 전체 환경이 상시 동작, 장애 시 확장 + DNS 전환	초~분 / 분	$$$
멀티 사이트 액티브-액티브	여러 리전이 동시에 트래픽 처리(Aurora Global, DynamoDB Global Tables, Route 53 지연/가중치)	거의 0 / 거의 0	$$$$

AWS Elastic Disaster Recovery(DRS)는 서버 블록 수준 지속 복제로 온프레미스/타 리전 서버의 DR을 저비용으로 구성합니다. **DR 훈련**을 정기적으로 하지 않은 DR 계획은 동작하지 않는다고 가정하세요. **AWS Fault Injection Service**로 장애를 주입해 검증할 수 있습니다.

15.3 비용 구조와 최적화

데이터 전송 요금 — 가장 헷갈리는 부분

흐름	요금(일반적)
인터넷 → AWS (인바운드)	무료
AWS → 인터넷 (아웃바운드)	GB당 과금(월 100 GB 무료 포함 구간 있음) — CloudFront 경유가 대개 저렴
같은 AZ 내 사설 IP 통신	무료
AZ 간 통신	양방향 GB당 과금 (RDS Multi-AZ 복제는 무료)
리전 간	GB당 과금
NAT Gateway	시간 + 처리 GB당 (+ 이후 인터넷 전송 요금)
VPC 엔드포인트(게이트웨이)	무료
TGW	Attachment 시간 + 처리 GB당

비용 최적화 체크리스트

1. **가시화**: 비용 할당 태그, Cost Explorer, Budgets, Cost Anomaly Detection, CUR(상세 보고서).
2. **Right-sizing**: Compute Optimizer 권고 반영, 과대 인스턴스 축소.
3. **약정**: 안정 사용량에 Savings Plans.
4. **스팟**: 배치·CI·상태 비저장 워커.
5. **Graviton** 전환, gp2 → gp3 전환.
6. **스케줄**: 개발 환경은 업무 시간 외 중지(Instance Scheduler).
7. **스토리지 계층화**: S3 수명 주기·Intelligent-Tiering, 오래된 스냅샷 정리.
8. **네트워크**: S3/DynamoDB 게이트웨이 엔드포인트, AZ 간 트래픽 최소화, CloudFront로 아웃바운드 절감, 미사용 EIP·퍼블릭 IPv4 정리.
9. **서버리스·관리형**으로 유휴 비용 제거.
10. **로그 보존 기간** 설정(CloudWatch Logs 무기한 보존 방지).

CHAPTER 16

실습: 3-Tier 웹 아키텍처 구축

학습 목표 · 지금까지 배운 리소스를 직접 엮어 동작하는 서비스를 만든다 · 패킷이 어떤 리소스를 거치는지 한 홉씩 설명한다 · 실습 후 모든 리소스를 정리한다

16.1 목표 아키텍처

```
사용자 ──HTTPS──▶ Route 53 (app.example.com, Alias)
                    │
                    ▼
              CloudFront (+ WAF)        ← 선택 사항
                    │
                    ▼
     ALB (퍼블릭 서브넷 a/c, sg-alb: 443 from 0.0.0.0/0)
                    │  HTTP 8080
                    ▼
     ASG: EC2 (프라이빗 앱 서브넷 a/c, sg-app: 8080 from sg-alb)
        │ 아웃바운드 업데이트 → NAT GW(같은 AZ) → IGW
        │ S3 접근 → 게이트웨이 엔드포인트
        │ 3306
        ▼
     RDS MySQL Multi-AZ (프라이빗 DB 서브넷 a/c, sg-db: 3306 from sg-app)
운영 접속: SSM Session Manager (SSH 포트 없음, 배스천 없음)
```

16.2 단계별 CLI 실습

아래 명령은 흐름 이해를 위한 것입니다. 반복 실습은 14장의 Terraform 코드로 하는 것을 권장합니다. ID 값은 앞 단계 출력에서 받아 변수에 저장합니다.

1단계: VPC와 서브넷

```
export AWS_DEFAULT_REGION=ap-northeast-2
VPC=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=lab-vpc}]' \
  --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --vpc-id $VPC --enable-dns-hostnames
mk() { aws ec2 create-subnet --vpc-id $VPC --cidr-block $1 --availability-zone $2 \
       --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=$3}]" \
       --query Subnet.SubnetId --output text; }
PUB_A=$(mk 10.0.1.0/24  ap-northeast-2a pub-a);  PUB_C=$(mk 10.0.2.0/24  ap-northeast-2c pub-c)
APP_A=$(mk 10.0.11.0/24 ap-northeast-2a app-a);  APP_C=$(mk 10.0.12.0/24 ap-northeast-2c app-c)
DB_A=$(mk 10.0.21.0/24  ap-northeast-2a db-a);   DB_C=$(mk 10.0.22.0/24  ap-northeast-2c db-c)
```

2단계: 인터넷 게이트웨이와 퍼블릭 라우팅

```
IGW=$(aws ec2 create-internet-gateway --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
RT_PUB=$(aws ec2 create-route-table --vpc-id $VPC --query RouteTable.RouteTableId --output text)
aws ec2 create-route --route-table-id $RT_PUB --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
aws ec2 associate-route-table --route-table-id $RT_PUB --subnet-id $PUB_A
aws ec2 associate-route-table --route-table-id $RT_PUB --subnet-id $PUB_C
```

3단계: NAT Gateway와 프라이빗 라우팅 (AZ별)

```
for AZ in A C; do
  PUB=$(eval echo \$PUB_$AZ); APP=$(eval echo \$APP_$AZ)
  EIP=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
  NAT=$(aws ec2 create-nat-gateway --subnet-id $PUB --allocation-id $EIP \
        --query NatGateway.NatGatewayId --output text)
  aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT
  RT=$(aws ec2 create-route-table --vpc-id $VPC --query RouteTable.RouteTableId --output text)
  aws ec2 create-route --route-table-id $RT --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT
  aws ec2 associate-route-table --route-table-id $RT --subnet-id $APP
done
# DB 서브넷은 별도 라우트 테이블(기본 경로 없음) = 격리 서브넷
```

4단계: 보안 그룹 체이닝

```
sg() { aws ec2 create-security-group --group-name $1 --description $1 --vpc-id $VPC \
       --query GroupId --output text; }
SG_ALB=$(sg lab-alb); SG_APP=$(sg lab-app); SG_DB=$(sg lab-db)
aws ec2 authorize-security-group-ingress --group-id $SG_ALB --protocol tcp --port 80  --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $SG_ALB --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $SG_APP --protocol tcp --port 8080 --source-group $SG_ALB
aws ec2 authorize-security-group-ingress --group-id $SG_DB  --protocol tcp --port 3306 --source-group $SG_APP
```

5단계: 앱 서버 (Launch Template + ASG)

```
# 사전 준비: AmazonSSMManagedInstanceCore 정책이 붙은 역할 + 인스턴스 프로파일 "lab-ec2"
AMI=$(aws ssm get-parameter --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-arm64 \
      --query Parameter.Value --output text)
cat > userdata.sh <<'EOF'
#!/bin/bash
dnf -y install python3
mkdir -p /srv && cd /srv
echo "ok from $(hostname)" > health
echo "hello from $(hostname)" > index.html
nohup python3 -m http.server 8080 &
EOF
aws ec2 create-launch-template --launch-template-name lab-lt --launch-template-data "{
  \"ImageId\":\"$AMI\",\"InstanceType\":\"t4g.micro\",
  \"IamInstanceProfile\":{\"Name\":\"lab-ec2\"},
  \"SecurityGroupIds\":[\"$SG_APP\"],
  \"MetadataOptions\":{\"HttpTokens\":\"required\"},
  \"UserData\":\"$(base64 -w0 userdata.sh)\"}"
```

6단계: ALB와 대상 그룹

```
TG=$(aws elbv2 create-target-group --name lab-tg --protocol HTTP --port 8080 --vpc-id $VPC \
     --health-check-path /health --query 'TargetGroups[0].TargetGroupArn' --output text)
ALB=$(aws elbv2 create-load-balancer --name lab-alb --subnets $PUB_A $PUB_C \
      --security-groups $SG_ALB --query 'LoadBalancers[0].LoadBalancerArn' --output text)
aws elbv2 create-listener --load-balancer-arn $ALB --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn=$TG
# (도메인이 있다면) ACM 인증서 발급 후 443 리스너 추가, 80은 리다이렉트로 변경
aws autoscaling create-auto-scaling-group --auto-scaling-group-name lab-asg \
  --launch-template LaunchTemplateName=lab-lt --min-size 2 --max-size 4 --desired-capacity 2 \
  --vpc-zone-identifier "$APP_A,$APP_C" --target-group-arns $TG \
  --health-check-type ELB --health-check-grace-period 120
aws autoscaling put-scaling-policy --auto-scaling-group-name lab-asg --policy-name cpu50 \
  --policy-type TargetTrackingScaling --target-tracking-configuration \
  '{"PredefinedMetricSpecification":{"PredefinedMetricType":"ASGAverageCPUUtilization"},"TargetValue":50}'
```

7단계: 확인

```
DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB --query 'LoadBalancers[0].DNSName' --output
text)
for i in 1 2 3 4; do curl -s http://$DNS/; done        # 두 호스트 이름이 번갈아 나오면 성공
aws elbv2 describe-target-health --target-group-arn $TG  # healthy 확인
```

RDS(8단계)는 `aws rds create-db-subnet-group` (DB_A, DB_C) → `aws rds create-db-instance --multi-az --no-publicly-accessible --vpc-security-group-ids $SG_DB --manage-master-user-password` (비밀번호를 Secrets Manager가 관리) 순으로 생성합니다.

16.3 패킷 흐름 따라가기 (면접 단골 질문)

ℹ 사용자 요청 한 건이 거치는 경로

① 브라우저가 `app.example.com` 조회 → Route 53이 ALB의 IP들 반환(Alias)
② 클라이언트 → ALB 퍼블릭 IP:80. 인터넷에서 **IGW**가 목적지 공인 IP를 ALB 노드 ENI의 사설 IP로 변환
③ **퍼블릭 서브넷 NACL** 인바운드 평가 → **sg-alb** 인바운드 443/80 허용
④ ALB가 대상 선택(헬스 체크 정상인 인스턴스). 새 TCP 연결: 출발지 = ALB 노드 사설 IP, 목적지 = 10.0.11.x:8080
⑤ VPC 라우트 테이블 `local` 경로 → 앱 서브넷 NACL 인바운드 → **sg-app**(소스 sg-alb, 8080 허용)
⑥ 앱이 DB 조회: 10.0.11.x → 10.0.21.y:3306, `local` 경로, **sg-db**(소스 sg-app) 허용
⑦ 응답은 역순. SG는 상태 저장이라 자동 허용, NACL은 임시 포트 아웃바운드 규칙 필요
⑧ 앱 서버가 OS 업데이트 시: 0.0.0.0/0 → **같은 AZ NAT GW**(사설 IP를 EIP로 SNAT) → IGW → 인터넷

16.4 일부러 망가뜨려 보기 (트러블슈팅 훈련)

이렇게 바꿔보세요	예상 증상 / 확인할 것
sg-app의 소스를 sg-alb → 다른 SG로 변경	대상 unhealthy → ALB 503, Flow Logs에 REJECT
앱 서브넷 NACL에 아웃바운드 1024-65535 거부	헬스 체크 실패(응답이 못 나감)
app-a 라우트 테이블에서 NAT 경로 삭제	app-a 인스턴스만 `dnf update` 타임아웃, SSM 접속도 실패(엔드포인트가 없다면)
헬스 체크 경로를 /nope으로	모든 대상 unhealthy (404) → 503
인스턴스 하나에서 웹 서버 프로세스 종료	ELB 헬스 체크 유형이면 ASG가 자동 교체

Reachability Analyzer로 "ALB ENI → 인스턴스 ENI:8080" 경로를 분석해 어느 구성 요소가 막는지 확인해 보세요.

16.5 정리 (반드시!)

```
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name lab-asg --force-delete
aws elbv2 delete-load-balancer --load-balancer-arn $ALB
aws elbv2 delete-target-group --target-group-arn $TG   # LB 삭제 후 잠시 뒤
aws ec2 delete-launch-template --launch-template-name lab-lt
# NAT GW 삭제 → 삭제 완료 대기 → EIP 해제 (해제 안 하면 계속 과금)
# RDS 삭제(--skip-final-snapshot 또는 최종 스냅샷), SG, 서브넷, 라우트 테이블, IGW 분리/삭제, VPC 삭제
# 마지막으로 Cost Explorer / Tag Editor에서 Project=study 리소스가 남았는지 확인
```

CHAPTER 17

트러블슈팅 가이드

학습 목표 · "연결이 안 된다"를 계층적으로 좁혀 원인을 찾는다 · 증상별 1순위 의심 항목을 안다

17.1 연결 문제 진단 순서 (바깥에서 안으로, 아래에서 위로)

1. **DNS**: 이름이 올바른 IP로 풀리나? `dig app.example.com`, 프라이빗 호스팅 영역이 VPC에 연결되었나?
2. **라우팅**: 출발지 서브넷 라우트 테이블에 목적지 경로가 있나? 돌아오는 경로도 있나(비대칭 라우팅)? Peering/TGW는 **양쪽 모두** 경로 필요.
3. **NACL**: 양쪽 서브넷, 양방향(임시 포트 포함) 허용인가?
4. **보안 그룹**: 목적지 인바운드 허용? 출발지 아웃바운드 허용(기본 전체 허용을 지웠다면)?
5. **대상 OS**: 프로세스가 `0.0.0.0`에서 리슨 중인가(127.0.0.1만 리슨하면 외부 불가)? OS 방화벽(firewalld/iptables)?
6. **애플리케이션**: 로그, 헬스 체크 응답 코드.

```
# 인스턴스 안에서 쓰는 기본 도구
ss -tlnp                       # 어떤 포트를 어디서 리슨하는지
curl -v telnet://10.0.21.8:3306   # TCP 연결 테스트 (또는 nc -zv host port)
dig +short myservice.internal  # DNS 해석
traceroute -T -p 443 example.com
# "Connection refused" = 패킷은 도달했으나 리슨하는 프로세스가 없음 (네트워크는 OK)
# "Timeout"           = 중간에서 버려짐 → SG/NACL/라우팅/NAT를 의심
```

17.2 증상별 체크리스트

증상	1순위 의심
EC2에 SSH/RDP 타임아웃	퍼블릭 IP 없음, 서브넷에 IGW 경로 없음, SG에 내 IP 미허용, NACL. → 가능하면 Session Manager 사용
프라이빗 EC2가 인터넷 불가	라우트 테이블의 NAT 경로, NAT GW가 **퍼블릭** 서브넷에 있는지, NAT 서브넷의 IGW 경로
ALB 503	대상 그룹에 healthy 대상 없음 → 헬스 체크 경로/포트/SG
ALB 504	대상 응답 지연(유휴 타임아웃), 대상 SG가 ALB 차단, 앱 무응답
ALB 502	대상이 연결을 끊음(keep-alive 타임아웃이 ALB보다 짧음), 잘못된 응답, TLS 불일치
S3 403 AccessDenied	IAM 정책, 버킷 정책 Deny(VPCE/IP 조건), KMS 키 정책 권한 누락, 퍼블릭 액세스 차단, 객체 소유권
인터페이스 엔드포인트 타임아웃	엔드포인트 SG 443 인바운드, 프라이빗 DNS 활성화, VPC DNS 속성 두 개
피어링 후 통신 불가	양쪽 라우트 테이블, SG에 상대 CIDR/SG, 전이 라우팅 시도 여부
VPN 터널 UP인데 통신 불가	라우트 전파/정적 경로, 온프레미스 방화벽, 암호화 도메인(정책 기반 VPN) 불일치
RDS 연결 불가	SG 3306/5432, 클라이언트가 VPC 내부인지, 엔드포인트 이름 사용 여부(IP 하드코딩 금지)
Lambda가 VPC 연결 후 외부 API 실패	Lambda가 프라이빗 서브넷 + NAT 경로가 있는지
갑자기 "권한 없음"	CloudTrail에서 errorCode 확인, SCP 변경, 권한 경계, 세션 만료
요청이 스로틀링 (429/ThrottlingException)	API 한도, Service Quotas, 지수 백오프 재시도

> 💡 **진단 도구 모음**
>
> VPC Reachability Analyzer(경로 분석) · VPC Flow Logs(ACCEPT/REJECT) · CloudTrail(누가 바꿨나) · Config 타임라인(언제부터 바뀌었나) · ELB Access Logs · CloudWatch 지표(`HealthyHostCount` , `TargetResponseTime` , `HTTPCode_ELB_5XX_Count`) · IAM Policy Simulator · `aws sts get-caller-identity`

APPENDIX

부록

A. 핵심 용어집

용어	뜻
AZ	가용 영역. 리전 안의 독립된 데이터센터 묶음
CIDR	IP 대역 표기법(예: 10.0.0.0/16)
ENI	가상 네트워크 인터페이스
EIP	고정 공인 IPv4 주소
IGW	VPC와 인터넷을 연결하는 게이트웨이
NAT GW	프라이빗 서브넷의 아웃바운드 인터넷 접근용 관리형 NAT
SG / NACL	ENI 단위 상태 저장 방화벽 / 서브넷 단위 상태 비저장 방화벽
VGW / CGW	AWS 측 VPN 게이트웨이 / 고객 측 게이트웨이 정보
TGW	여러 VPC·온프레미스를 잇는 허브 라우터
DX / VIF	Direct Connect 전용선 / 그 위의 가상 인터페이스(VLAN+BGP)
BGP / ASN	동적 라우팅 프로토콜 / 자율 시스템 번호
PrivateLink	인터페이스 엔드포인트로 서비스를 사설 연결하는 기술
ARN	AWS 리소스 고유 식별자
STS	임시 자격 증명 발급 서비스
SCP	Organizations에서 계정 최대 권한을 제한하는 정책
RTO / RPO	복구 목표 시간 / 복구 목표 시점(데이터 손실 허용)
IaC	코드로 인프라를 정의·배포
IMDS	인스턴스 메타데이터 서비스(169.254.169.254)
OAC	CloudFront만 S3 오리진에 접근하게 하는 설정
Hyperplane	NAT GW, NLB, PrivateLink, Lambda VPC ENI 등의 기반인 AWS 내부 분산 네트워크 기능

B. 자주 쓰는 포트 번호

포트	프로토콜	포트	프로토콜
22	SSH / SFTP	3306	MySQL / MariaDB / Aurora MySQL
25 / 587 / 465	SMTP (EC2 25번은 기본 제한)	5432	PostgreSQL
53	DNS (UDP/TCP)	1433	SQL Server
80 / 443	HTTP / HTTPS	1521	Oracle
123	NTP (AWS: 169.254.169.123)	6379	Redis / Valkey
179	BGP	11211	Memcached
389 / 636	LDAP / LDAPS	27017	MongoDB / DocumentDB
500 / 4500 (UDP)	IPsec IKE / NAT-T	2049	NFS (EFS)
3389	RDP	445	SMB (FSx Windows)
6081 (UDP)	GENEVE (GWLB)	9092	Kafka

C. CIDR 빠른 참조표

프리픽스	마스크	주소 수	프리픽스	마스크	주소 수
/16	255.255.0.0	65,536	/24	255.255.255.0	256
/17	255.255.128.0	32,768	/25	255.255.255.128	128
/18	255.255.192.0	16,384	/26	255.255.255.192	64
/19	255.255.224.0	8,192	/27	255.255.255.224	32
/20	255.255.240.0	4,096	/28	255.255.255.240	16
/21	255.255.248.0	2,048	/29	255.255.255.248	8
/22	255.255.252.0	1,024	/30	255.255.255.252	4
/23	255.255.254.0	512	/32	255.255.255.255	1

D. 주요 기본 한도 (변경 가능, Service Quotas에서 확인)

항목	기본값
리전당 VPC	5
VPC당 서브넷	200
VPC당 IPv4 CIDR 블록	5
리전당 Elastic IP	5
ENI당 보안 그룹	5 (최대 16)
보안 그룹당 인바운드/아웃바운드 규칙	각 60
NACL당 규칙	방향별 20 (최대 40)
라우트 테이블당 경로	50 (최대 1,000)
VPC당 활성 피어링	50 (최대 125)
VGW BGP 광고 수신 경로	100

E. 학습 로드맵과 자격증

단계	자격증 / 활동
입문	AWS Certified Cloud Practitioner (CLF-C02) — 개념 위주
핵심 (이 교재 범위)	**Solutions Architect – Associate (SAA-C03)** — 인프라 엔지니어 필수
운영/자동화	CloudOps Engineer – Associate(구 SysOps), DevOps Engineer – Professional
네트워크 전문	**Advanced Networking – Specialty (ANS-C01)** — 5장 심화 + BGP/DX/하이브리드 DNS
설계 전문	Solutions Architect – Professional (SAP-C02)
보안	Security – Specialty (SCS-C03)

추천 학습 자료: AWS 공식 문서(특히 VPC User Guide, "How Amazon VPC works"), AWS Skill Builder(무료 강의·실습), AWS Well-Architected 문서, AWS re:Invent 네트워크 세션(NET3xx/4xx — "Advanced VPC design" 류), AWS Workshops(workshops.aws).

학습 팁: ① 그림을 손으로 그리기 ② 콘솔 → CLI → IaC 순서로 같은 것을 세 번 만들기 ③ 일부러 망가뜨리고 고치기 ④ Flow Logs와 CloudTrail을 직접 읽어보기 ⑤ 매주 What's New(aws.amazon.com/new) 훑어보기.

F. 확인 문제 정답

1장

1) 최소 2개 AZ에 분산 배치(ELB + ASG 등). 2) EC2 위 MySQL은 고객, RDS는 AWS가 엔진 패치(적용 시점·유지 관리 기간 설정은 고객). 3) AZ 이름이 아닌 AZ ID(apne2-az1 등).

2장

1) 1,024개, 사용 가능 1,019개. 2) 10.0.8.0/21은 올바름(8의 배수), 10.0.12.0/21은 무효(/21의 시작은 0, 8, 16…). 3) 더 긴 /16이 이기므로 pcx(피어링). 4) 안 됨. 응답은 클라이언트의 임시 포트로 나가야 하므로 아웃바운드 1024-65535 허용 필요.

3장

1) 불가 — 명시적 Deny 우선. 2) (a) Lambda 실행 역할에 s3:GetObject 허용 + 계정 B 버킷 정책에서 계정 A의 역할 허용, (b) 계정 B에 읽기 역할을 만들고 Lambda가 AssumeRole. (KMS 암호화면 키 정책도) 3) 불가 — SCP는 권한을 주지 않으며 Allow가 별도로 필요.

4장

1) 안 됨 — 퍼블릭 IP/EIP 필요. 2) AZ-a 장애 시 모든 프라이빗 서브넷의 아웃바운드가 끊기고, 다른 AZ에서 AZ 간 전송 요금 발생 → AZ별 NAT GW(또는 리전 NAT GW). 3) NACL — SG는 거부 규칙이 없음. 4) 앱 서버의 보안 그룹 ID.

5장

1) A–C를 직접 피어링하거나 Transit Gateway 사용(피어링은 전이 불가). 2) S3 게이트웨이 엔드포인트(무료) 추가. 3) Route 53 Resolver 인바운드 엔드포인트 + 온프레미스 DNS에 조건부 전달 설정(+ VPN/DX 연결). 4) Direct Connect.

6장

1) 스팟 인스턴스(여러 인스턴스 유형, 체크포인트). 2) 스냅샷 생성 → AZ-c에서 볼륨 복원 → 연결. 3) ASG 헬스 체크 유형이 ELB인지, 대상 그룹 헬스 체크 설정. 4) Lambda를 프라이빗 서브넷에 두고 NAT GW 경로 설정(퍼블릭 서브넷에 둬도 퍼블릭 IP를 받지 못함).

8장

1) 0~30일 Standard → 30일 Standard-IA(또는 Glacier IR) → 365일 Glacier Flexible/Deep Archive → 7년(2,555일) 후 만료 삭제. 2) EFS. 3) CloudFront + OAC + ACM 인증서, 버킷은 퍼블릭 액세스 차단 유지.

9장

1) 아니요 — Multi-AZ 스탠바이는 읽기 불가(인스턴스 배포 기준). 읽기 전용 복제본 또는 Aurora 리더로 분리. 2) Aurora Global Database(또는 DynamoDB Global Tables). 3) RDS Proxy.

> **마치며**
>
> AWS 네트워크를 잘 아는 엔지니어와 그렇지 않은 엔지니어의 차이는 결국 **"패킷 하나가 어디를 거쳐 어떻게 돌아오는지 설명할 수 있는가"**입니다. 16장의 패킷 흐름을 보지 않고 설명할 수 있을 때까지 반복해 보세요.
> 서비스의 세부 수치(한도, 가격, 성능)는 자주 바뀌므로, 원리를 이해하고 숫자는 공식 문서에서 확인하는 습관을 들이세요.